

Unit 6: Database Maintenance and Performance Management

Overview of Database Maintenance

- Database maintenance refers to the set of activities performed to ensure the efficient and reliable operation of a database system. It involves tasks aimed at preventing data corruption, optimizing performance, ensuring data availability, and managing resources effectively.
- Database maintenance refers to a set of practices aimed at keeping your database healthy, efficient, and performing optimally over time

View the Alert Log

The alert log is an XML file that is a chronological log of messages and errors.

There is one alert log in each ADR home. Each alert log is specific to its component type, such as database, Oracle ASM, listener, and Oracle Clusterware.

For the database, the alert log includes messages about the following:

- Critical errors (incidents)
- Administrative operations, such as starting up or shutting down the database, recovering the database, creating or dropping a tablespace, and others.
- Errors during automatic refresh of a materialized view
- Other database events

You can view the alert log in text format (with the XML tags stripped) with Cloud Control and with the ADRCI utility. There is also a text-formatted version of the alert log stored in the ADR for backward compatibility. However, Oracle recommends that any parsing of the alert log contents be done with the XML-formatted version, because the text format is unstructured and may change from release to release

You can view the alert log with a text editor, with Cloud Control, or with the ADRCI utility.

To view the alert log with Cloud Control:

1. Access the Database Home page in Cloud Control.
2. From the Oracle Database menu, select **Diagnostics**, then **Support Workbench**.
3. Under Related Links, click **Alert Log Contents**.
The View Alert Log Contents page appears.
4. Select the number of entries to view, and then click **Go**.

To view the alert log with a text editor:

1. Connect to the database with SQL*Plus or another query tool, such as SQL Developer.
2. Query the V\$DIAG_INFO view as shown in "Viewing ADR Locations with the V\$DIAG_INFO View".
3. To view the text-only alert log, without the XML tags, complete these steps:
 - a. In the V\$DIAG_INFO query results, note the path that corresponds to the Diag Trace entry, and change directory to that path.
 - b. Open file alert_*SID*.log with a text editor.
4. To view the XML-formatted alert log, complete these steps:
 - a. In the V\$DIAG_INFO query results, note the path that corresponds to the Diag Alert entry, and change directory to that path.
 - b. Open the file log.xml with a text editor.

Viewing ADR Locations with the V\$DIAG_INFO View

The V\$DIAG_INFO view lists all important ADR locations for the current Oracle Database instance.

```
SELECT * FROM V$DIAG_INFO;
```

INST_ID	NAME	VALUE

1	Diag Enabled	TRUE
1	ADR Base	/u01/oracle
1	ADR Home	/u01/oracle/diag/rdbms/orclbi/orclbi
1	Diag Trace	/u01/oracle/diag/rdbms/orclbi/orclbi/trace
1	Diag Alert	/u01/oracle/diag/rdbms/orclbi/orclbi/alert
1	Diag Incident	/u01/oracle/diag/rdbms/orclbi/orclbi/incident
1	Diag Cdump	/u01/oracle/diag/rdbms/orclbi/orclbi/cdump
1	Health Monitor	/u01/oracle/diag/rdbms/orclbi/orclbi/hm
1	Default Trace File	/u01/oracle/diag/rdbms/orclbi/orclbi/trace/orcl_ora_22769.trc
1	Active Problem Count	8
1	Active Incident Count	20

Data in the V\$DIAG_INFO View

The following table describes some of the information displayed by this view.

Name	Description
ADR Base	Path of ADR base
ADR Home	Path of ADR home for the current database instance
Diag Trace	Location of background process trace files, server process trace files, SQL trace files, and the text-formatted version of the alert log
Diag Alert	Location of the XML-formatted version of the alert log
Default Trace File	Path to the trace file for the current session

The Automatic Workload Repository (AWR)

The Automatic Workload Repository (AWR) in Oracle is a built-in feature that collects and maintains performance statistics for the database. It's a vital component of Oracle's self-diagnostic and tuning capabilities, particularly in the Oracle Database Enterprise Edition.

- ☞ AWR is a monitoring tool that collects, analyzes, and stores performance statistics, helping database administrators identify and resolve performance problems. When tuning a database, each component has its own unique set of statistics. To provide a common scale for comparison, database performance statistics are reported in a database-timed model. This model offers details on how much time is spent on performing database activities.
- AWR collects, processes, and maintains performance statistics for problem detection and self-tuning purposes. This gathered data is stored both in memory and in the database, and is displayed in both reports and views.

The statistics collected and processed by AWR include:

- Object statistics that determine both access and usage statistics of database segments
- Time model statistics based on time usage for activities, displayed in the V\$SYS_TIME_MODEL and V\$SESS_TIME_MODEL views
- Some of the system and session statistics collected in the V\$SYSSTAT and V\$SESSTAT views
- SQL statements that are producing the highest load on the system, based on criteria such as elapsed time and CPU time

- Active Session History (ASH) statistics, representing the history of recent sessions activity

Snapshots

- ☞ Snapshots are sets of historical data for specific time periods that are used for performance comparisons by Automatic Database Diagnostic Monitor (ADDM). By default, Oracle Database automatically generates snapshots of the performance data once every hour and retains the statistics in AWR for 8 days. You can also manually create snapshots or change the snapshot retention period, but it is usually not necessary.
- ☞ AWR compares the difference between snapshots to determine which SQL statements to capture based on the effect on the system load. This reduces the number of SQL statements that must be captured over time. After the snapshots are created, ADDM analyzes the data captured in the snapshots to perform its performance analysis.

Baselines

- ☞ A baseline is a set of snapshots from a specific time period that is preserved for comparison with other snapshots when a performance problem occurs. The snapshots contained in a baseline are excluded from the automatic AWR purging process and are retained indefinitely.
- ☞ There are several types of available baselines:
 - Fixed Baselines
 - Moving Window Baselines
 - Baseline Templates
- ☞ A **fixed baseline** corresponds to a fixed, contiguous time period in the past that you specify. Before creating a fixed baseline, carefully consider the time period you choose as a baseline, because the baseline should represent the system operating at an optimal level. In the future, you can compare the baseline with other baselines or snapshots captured during periods of poor performance to analyze performance degradation over time.

- ☞ A **moving window baseline** corresponds to all AWR data that exists within the AWR retention period. This is useful when using adaptive thresholds because the database can use AWR data in the entire AWR retention period to compute metric threshold values.

Oracle Database automatically maintains a system-defined moving window baseline. The default window size for the system-defined moving window baseline is the current AWR retention period, which by default is 8 days. If you are planning to use adaptive thresholds, then consider using a larger moving window—such as 30 days—to accurately compute threshold values. You can resize the moving window baseline by changing the number of days in the moving window to a value that is equal to or less than the number of days in the AWR retention period. Therefore, to increase the size of a moving window, you must first increase the AWR retention period accordingly.

- **Baseline templates** enable you to create baselines for a contiguous time period in the future. There are two types of baseline templates:
 - Single Baseline Templates
 - Repeating Baseline Templates
- Use a single baseline template to create a baseline for a single contiguous time period in the future. This is useful if you know beforehand of a time period that you intend to capture in the future. For example, you may want to capture AWR data during a system test that is scheduled for the upcoming weekend. In this case, you can create a single baseline template to automatically capture the time period when the test occurs.
- Use a repeating baseline template to create and drop baselines based on a repeating time schedule. This is useful if you want Oracle Database to automatically capture a contiguous time period on an ongoing basis. For example, you may want to capture AWR data during every Monday morning for a month. In this case, you can create a repeating baseline template to automatically create baselines on a repeating schedule for every Monday, and automatically remove older baselines after a specified expiration interval, such as one month.

Space Consumption

The space consumed by AWR is determined by several factors:

- Number of active sessions in the database at any given time
- Snapshot interval

The snapshot interval determines the frequency at which snapshots are captured. A smaller snapshot interval increases the frequency, which increases the volume of data collected by AWR.

- Historical data retention period

The retention period determines how long this data is retained before being purged. A longer retention period increases the space consumed by AWR.

By default, Oracle Database captures snapshots once every hour and retains them in the database for 8 days. With these default settings, a typical system with an average of 10 concurrent active sessions can require approximately 200 to 300 MB of space for its AWR data.

To reduce AWR space consumption, increase the snapshot interval and reduce the retention period. When reducing the retention period, note that several Oracle Database self-managing features depend on AWR data for proper functioning. Not having enough data can affect the validity and accuracy of these components and features, including:

- Automatic Database Diagnostic Monitor (ADDM)
- SQL Tuning Advisor
- Undo Advisor
- Segment Advisor

If possible, Oracle recommends that you set the AWR retention period large enough to capture at least one complete workload cycle. If your system experiences weekly workload cycles, such as OLTP workload during weekdays and batch jobs during the weekend, then you do not need to change the default AWR retention period of 8 days. However, if your system is subjected to a monthly peak load during month-end book closing, then you may need to set the retention period to one month.

Under exceptional circumstances, you can disable automatic snapshot collection by setting the snapshot interval to 0. Under this condition, the automatic collection of the workload and statistical data is stopped, and most of the Oracle Database self-management functionality is not operational. In addition, you cannot manually create snapshots. For this reason, Oracle strongly recommends against disabling automatic snapshot collection.

Adaptive Thresholds

- Adaptive thresholds enable you to monitor and detect performance issues, while minimizing administrative overhead. Adaptive thresholds automatically set warning and critical alert thresholds for some system metrics using statistics derived from metric values captured in the moving window baseline. The statistics for these thresholds are recomputed weekly and might result in new thresholds as system performance evolves over time. Additionally, adaptive thresholds can compute different thresholds values for different times of the day or week based on periodic workload patterns.

For example, many databases support an online transaction processing (OLTP) workload during the day and batch processing at night. The performance metric for response time per transaction can be useful for detecting degradation in OLTP performance during the day. However, a useful OLTP threshold value is usually too low for batch workloads, where long-running transactions might be common. As a result, threshold values appropriate to OLTP might trigger frequent false performance alerts during batch processing. Adaptive thresholds can detect such a workload pattern and automatically set different threshold values for daytime and nighttime.

- There are two types of adaptive thresholds:
 - *Percentage of Maximum Thresholds*
 - *Significance Level Thresholds*

Percentage of Maximum Thresholds

The threshold value for percentage of maximum thresholds is computed as a percentage multiple of the maximum value observed for the data in the moving window baseline.

Percentage of maximum thresholds are most useful when a system is sized for peak workloads, and you want to be alerted when the current workload volume approaches or exceeds previous high values. Metrics that have an unknown but definite limiting value are prime candidates for these settings. For example, the redo generated per second metric is typically a good candidate for a percentage of maximum threshold.

Significance Level Thresholds

The threshold value for significance level thresholds is set to a statistical percentile that represents how unusual it is to observe values above the threshold value based the data in the moving window baseline.

Significance level thresholds are most useful for metrics that exhibit statistically stable behavior when the system is operating normally, but might vary over a wide range when the system is performing poorly. For example, the response time per transaction metric should be stable for a well-tuned OLTP system, but may fluctuate widely when performance issues arise. Significance level thresholds are meant to generate alerts when conditions produce both unusual metric values and unusual system performance.

Significance level thresholds can be set to one of the following levels:

High (.95)

Only 5 in 100 observations are expected to exceed this value.

Very High (.99)

Only 1 in 100 observations are expected to exceed this value.

Severe (.999)

Only 1 in 1,000 observations are expected to exceed this value.

Extreme (.9999)

Only 1 in 10,000 observations are expected to exceed this value.

When you specify a significance level threshold, Oracle Database performs an internal calculation to set the threshold value. In some cases, Oracle Database cannot establish the threshold value at higher significance levels using the data in the baseline, and the significance level threshold is not set.

If you specified a Severe (.999) or Extreme (.9999) significance level threshold and are not receiving alerts as expected, try setting the significance level threshold to a lower value, such as Very High (.99) or High (.95). Alternatively, consider using a percentage of maximum threshold instead. If you change the threshold and find that you are receiving too many alerts, try increasing the number of occurrences to trigger an alert.

Here's how AWR generally works:

- **Data Collection:** The AWR automatically collects and stores performance-related data at regular intervals, typically every hour by default, although the interval can be modified.
- **Statistics Gathering:** It captures statistics related to system and session activity, object usage, and resource consumption, among other things. This data includes metrics like CPU usage, I/O rates, memory usage, SQL statement execution details, and wait events.
- **Storage:** AWR data is stored in the database's AWR repository, which is a set of database tables in the SYSAUX tablespace. The data is retained for a configurable retention period (usually 7 or 8 days by default) to allow for historical analysis.
- **Analysis and Reporting:** Database administrators (DBAs) can use Oracle's Enterprise Manager or command-line tools like SQL*Plus to generate AWR reports. These reports provide insights into database performance over time, identifying performance bottlenecks, trends, and areas for optimization.
- **Tuning and Optimization:** Based on the insights gained from AWR reports, DBAs can make informed decisions to optimize database performance. This might involve adjusting database parameters, SQL tuning, schema design changes, or hardware upgrades.

The AWR plays a crucial role in Oracle's proactive performance management strategy, helping DBAs identify and address performance issues before they impact the system's

availability or end-user experience. It's a powerful tool for maintaining the health and efficiency of Oracle databases in enterprise environments.

Accessing AWR Reports

☞ One can generate the AWR report either through **Oracle Enterprise Manager** or via **command line**.

Running AWR from Command Line

Running the Automatic Workload Repository (AWR) reports from the command line in Oracle involves using Oracle's command-line interfaces like SQL*Plus. Here's a step-by-step guide on how to do it:

1. **Connect to the Database:** First, open your command-line interface (like Command Prompt on Windows or Terminal on Unix/Linux) and connect to your Oracle database using SQL*Plus. You'll need the username, password, and service name (or SID) of the database.
2. **Generate AWR Report:** Once connected, you can generate an AWR report using the @ command in SQL*Plus. This command executes a script file that generates the AWR report.

After executing this command, you'll be prompted to enter the Report Type, Begin Snapshot ID, End Snapshot ID, and Report Name.

Report Type: Specify the type of report you want to generate. You can choose html for HTML format or text for plain text format.

Begin Snapshot ID: Enter the ID of the starting snapshot for the report.

End Snapshot ID: Enter the ID of the ending snapshot for the report.

Report Name: Provide a name for the report file.

3. **View the AWR Report:** Once you've provided the required information, the AWR report will be generated and saved in the directory specified by your command-line

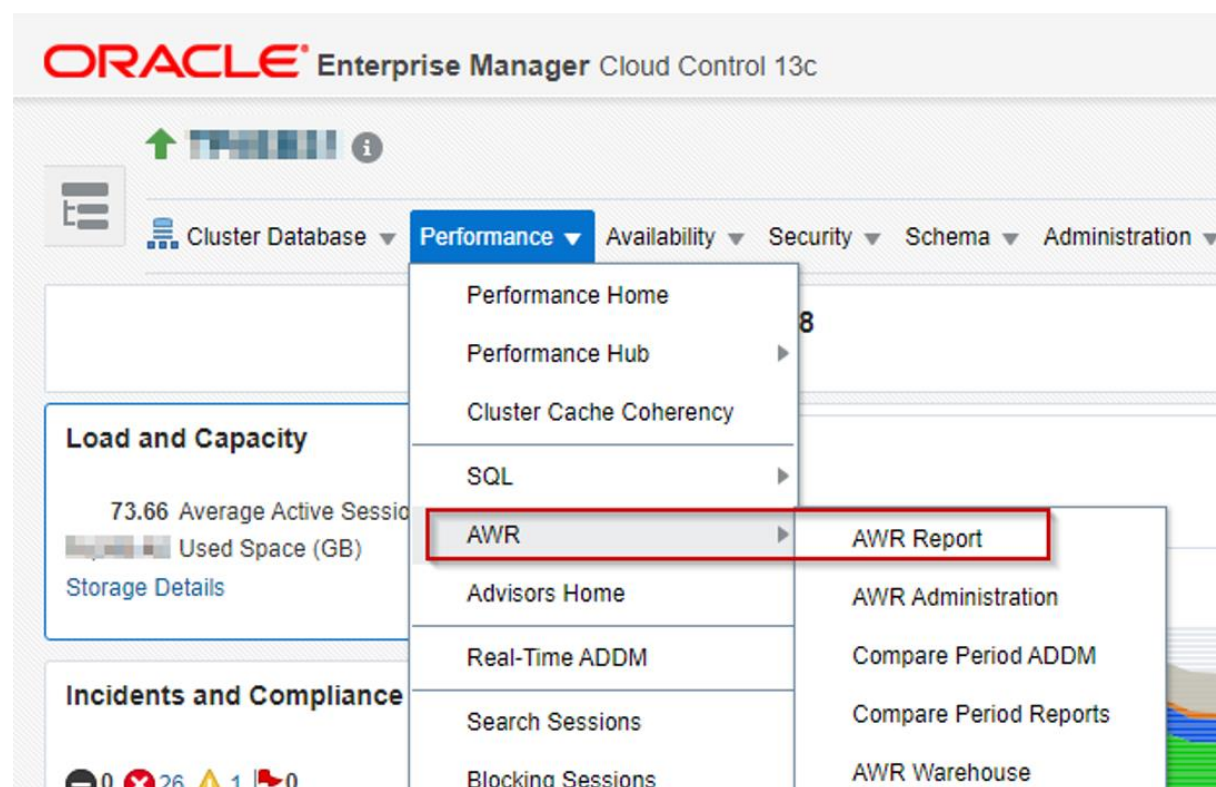
interface. You can open the report file using a text editor or a web browser, depending on the format you selected.

For example, if you chose HTML format, you can open the report file in a web browser to view the detailed performance analysis.

4. **Exit SQL*Plus:** After viewing the report, you can exit the SQL*Plus session by typing exit and pressing Enter.

Running AWR from Oracle Enterprise Manager

Access the AWR Report by navigating to Performance -> AWR -> AWR Report on the database page.



Click the search button, choose the Begin and End snapshots, then select Generate Report. Once the report is generated, click the Save to File button to save the report to your local drive.

Reading AWR Reports

- The AWR has multiple sections that cover various database activities. To find what you need quickly, you may want to navigate directly to the appropriate section. Important sections in the AWR are discussed below:

Top Timed Events

The Top Timed Events section provides an overview of CPU and I/O constraints and displays where the most time is spent waiting for resources. Further breakdowns are available in the Foreground and Background Events sections.

Top Timed Events

- Instance 's' - cluster wide summary
- 's' Waits, %Timeouts, Wait Time Total(s) : Cluster-wide total for the wait event
- 's' 'Wait Time Avg (ms)' : Cluster-wide average computed as (Wait Time Total / Event Waits) in ms
- 's' Summary 'Avg Wait Time (ms)' : Per-instance 'Wait Time Avg (ms)' used to compute the following statistics
- 's' [Avg/Min/Max/Std Dev] : average/minimum/maximum/standard deviation of per-instance 'Wait Time Avg(ms)'
- 's' Cnt : count of instances with wait times for the event

#	Wait		Event		Wait Time			Summary Avg Wait Time (ms)				
	Class	Event	Waits	%Timeouts	Total(s)	Avg(ms)	%DB time	Avg	Min	Max	Std Dev	Cnt
*		DB CPU			72,087.21		76.84					4
	User I/O	db file sequential read	24,193,973	0.00	11,489.44	0.47	12.25	0.47	0.46	0.49	0.01	4
	User I/O	db file scattered read	1,860,646	0.00	1,600.17	0.86	1.71	0.86	0.72	0.94	0.10	4
	Cluster	gc current block 2-way	5,242,448	0.00	1,126.70	0.21	1.20	0.21	0.19	0.25	0.03	4
	Cluster	gc current block 3-way	2,105,459	0.00	920.52	0.44	0.98	0.44	0.39	0.49	0.04	4
	System I/O	log file parallel write	869,120	0.00	745.54	0.86	0.79	0.86	0.76	1.02	0.13	4
	User I/O	db file parallel read	673,249	0.00	662.62	0.98	0.71	0.95	0.84	1.04	0.08	4
	User I/O	direct path read	939,490	0.00	547.74	0.58	0.58	0.59	0.50	0.70	0.09	4
	Cluster	gc cr block busy	319,963	0.00	545.24	1.70	0.58	1.72	1.63	1.81	0.08	4
	Other	gcs log flush sync	2,135,938	1.83	485.92	0.23	0.52	0.24	0.19	0.27	0.03	4

System I/O Statistics

In the System I/O Statistics section, you can view the overall system statistics and confirm if the I/O subsystem can handle the I/O demands made by the database. You can also verify the redo log switches and adjust the size of the redo logs to ensure that there are 3 to 4 switches per hour.

System Statistics

#	Logical Reads	Physical Reads	Physical Writes	Redo Size (k)	Block Changes	User Calls	Execs	Parses	Logons	Txns
1	4,661,189,771	26,767,378	922,390	3,335,395	19,266,895	2,047,352	10,900,740	1,879,625	37,953	95,739
2	4,863,061,121	29,746,079	986,862	1,411,634	5,331,714	2,709,672	25,302,580	2,283,779	36,222	62,261
3	4,357,934,357	16,129,132	1,413,711	4,783,648	29,865,621	1,843,681	21,046,087	1,705,415	30,301	77,217
4	4,226,095,332	33,186,415	768,292	8,134,120	53,381,033	2,630,268	27,064,889	1,873,478	26,517	98,247
Sum	18,108,280,581	105,829,004	4,091,255	17,664,798	107,845,263	9,230,973	84,314,296	7,742,297	130,993	333,464
Avg	4,527,070,145	26,457,251	1,022,814	4,416,200	26,961,316	2,307,743	21,078,574	1,935,574	32,748	83,366
Std	288,715,987	7,368,045	276,260	2,837,458	20,277,083	427,678	7,240,340	245,768	5,291	16,909

[Back to Global Activity Load Profile](#)

[Back to Top](#)

System Statistics - Per Second

#	Logical Reads/s	Physical Reads/s	Physical Writes/s	Redo Size (k/s)	Block Changes/s	User Calls/s	Execs/s	Parses/s	Logons/s	Txns/s
1	3,960,151.82	22,741.59	783.66	2,833.76	16,369.17	1,739.43	9,261.28	1,596.93	32.24	81.34
2	4,131,514.51	25,271.40	838.41	1,199.28	4,529.67	2,302.06	21,496.33	1,940.23	30.77	52.90
3	3,702,751.07	13,704.24	1,201.17	4,064.46	25,375.55	1,566.50	17,881.96	1,449.02	25.75	65.61
4	3,590,461.56	28,194.95	652.74	6,910.69	45,352.16	2,234.66	22,994.14	1,591.69	22.53	83.47
Sum	15,384,878.95	89,912.18	3,475.98	15,008.19	91,626.55	7,842.65	71,633.72	6,577.88	111.29	283.31
Avg	3,846,219.74	22,478.04	868.99	3,752.05	22,906.64	1,960.66	17,908.43	1,644.47	27.82	70.83
Std	245,200.00	6,259.26	234.75	2,410.71	17,227.38	363.29	6,151.19	208.74	4.49	14.37

SQL CPU & I/O Consumption

In this section, you can find details about the SQLs that were executed during the AWR interval. This section is important because it helps identify which SQLs were using the most system resources, leading to performance degradation. The SQLs with high consumption of CPU, I/O, or unoptimized reads are good candidates for tuning.

SQL ordered by Elapsed Time (Global)

• Captured SQL account for 47.1% of Total DB Time (s): 93.812
• Captured PUSQL account for 35.3% of Total DB Time (s): 93.812

SQL ID	Total							Per Execution							Percentage of Total							SQL Text						
	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Writes	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Writes	Cluster (s)	DB Time	DB CPU	IOWait (s)	Gets	Reads	Writes	Cluster		Execs					
9wksv5v55p6	3,689.93	3,689.93	0.06	1,313,170.903	0	0	4.35	185	19.95	19.95	0.00	7,098,221.10	0.00	0.00	0.02	3.93	5.01	0.00	7.25	0.00	0.09	0.00	0.00	SELECT ATTR_NO, ATTR_NAME, ATT...				
9wksv5v55p6	2,353.18	2,272.99	40.90	303,662.065	79.115	0	9.46	0	0	0	0	0	0	0	2.51	3.15	0.00	1.68	0.07	0.20	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)				
9wksv5v55p6	2,274.00	1,786.68	76.15	197,102.418	123.474	4.718	312.50	4.718	0.48	0.38	0.02	41,776.69	26.17	1.00	0.07	2.42	2.48	0.01	1.09	0.12	6.65	0.01	0.00	begin WITH_EVENT setNavigationPa...				
9wksv5v55p6	1,835.97	1,442.72	340.43	227,503.704	858.482	4	11.83	4	450.99	369.88	85.11	56,875,926.00	214,623.00	1.00	2.96	1.96	2.00	0.02	1.26	0.81	0.25	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)			
9wksv5v55p6	1,826.21	1,434.19	339.61	227,111.738	858.415	24.356	10.82	3	608.74	478.06	113.20	75,703,912.67	286,138.33	8.118.67	3.61	1.99	1.99	0.02	1.25	0.81	0.23	0.00	0.00	0.00	SELECT HTA.ATTRIBUTE1, HTA.AT...			
9wksv5v55p6	1,609.56	1,007.42	189.72	44,211.476	454.572	12	445.09	12	134.13	83.95	15.81	3,684,289.67	37,881.00	1.00	37.09	1.72	1.40	0.01	0.24	0.43	9.47	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)			
9wksv5v55p6	1,485.95	1,458.85	0.30	432,911.498	959	46	2.04	35	42.46	41.68	0.01	12,368,899.94	27.40	1.31	0.06	1.58	2.02	0.00	2.39	0.00	0.04	0.00	0.00	0.00	0.00	SELECT /*+ PARALLEL(32) */ RSH...		
9wksv5v55p6	1,484.02	1,457.32	0.59	432,682.849	1,115	33	2.08	33	44.99	44.16	0.02	13,111,601.48	33.79	1.00	0.06	1.58	2.02	0.00	2.39	0.00	0.04	0.00	0.00	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)	
9wksv5v55p6	1,268.21	832.74	20.97	11,191.191	21,916	2,321	445.24	12	105.68	69.39	1.75	932,599.25	1,828.33	193.42	37.10	1.35	1.16	0.00	0.06	0.02	9.48	0.00	0.00	0.00	0.00	0.00	0.00	SELECT CCB.COUNTER_ID, CCB.CTR...
2fwr5v5v55p6	1,177.59	190.66	970.49	9,889,603	1,778.473	0	96.94	0	0	0	0	0	0	0	1.26	0.26	0.07	0.05	1.68	2.06	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT "AT" "ACTUAL_EFFORT", ...

[Back to SQL Statistics](#)

[Back to Top](#)

SQL ordered by CPU Time (Global)

• Captured SQL account for 44.9% of Total CPU Time (s): 72.087
• Captured PUSQL account for 35.6% of Total CPU Time (s): 72.087

SQL ID	Total							Per Execution							Percentage of Total							SQL Text											
	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Writes	Cluster (s)	Elapsed (s)	CPU (s)	IOWait (s)	Gets	Reads	Writes	Cluster (s)	DB CPU	DB time	IOWait	Gets	Reads	Writes	Cluster		Execs										
9wksv5v55p6	3,689.93	3,689.93	0.06	1,313,170.903	0	0	4.35	185	19.95	19.95	0.00	7,098,221.10	0.00	0.00	0.02	3.93	5.01	0.00	7.25	0.00	0.09	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT ATTR_NO, ATTR_NAME, ATT...			
9wksv5v55p6	2,272.99	2,253.18	40.90	303,662.065	79.115	0	9.46	0	0	0	0	0	0	0	3.15	2.51	0.00	1.68	0.07	0.20	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)			
9wksv5v55p6	1,786.68	2,274.00	76.15	197,102.418	123.474	4.718	312.50	4.718	0.38	0.48	0.02	41,776.69	26.17	1.00	0.07	2.48	2.42	0.01	1.09	0.12	6.65	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	begin WITH_EVENT setNavigationPa...		
9wksv5v55p6	1,458.85	1,455.95	0.30	432,911.498	959	46	2.04	35	42.46	41.68	0.01	12,368,899.94	27.40	1.31	0.06	1.58	2.02	0.00	2.39	0.00	0.04	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT /*+ PARALLEL(32) */ RSH...		
9wksv5v55p6	1,484.02	1,457.32	0.59	432,682.849	1,115	33	2.08	33	44.99	44.16	0.02	13,111,601.48	33.79	1.00	0.06	1.58	2.02	0.00	2.39	0.00	0.04	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)	
9wksv5v55p6	1,442.72	1,825.97	34.20	227,703.744	858.415	24.356	10.82	3	608.74	459.89	85.85	115,785,975.00	214,623.00	1.00	2.96	2.00	1.96	0.20	1.26	1.20	0.81	0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)	
9wksv5v55p6	1,434.19	1,260.21	339.61	227,703.744	858.415	24,356	10.82	3	678.06	474.17	113.20	75,703,912.67	286,138.33	8.118.67	3.61	1.99	1.99	0.02	1.25	0.81	0.23	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT HTA.ATTRIBUTE1, HTA.AT...	
9wksv5v55p6	1,109.54	1,176.59	0.32	263,577.468	11,601	0	0.01	0	0	0	0	0	0	0	1.62	1.25	0.00	1.46	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	BEGIN WITH (SELECT * FROM ...)	
9wksv5v55p6	1,169.28	1,176.04	0.00	263,561.112	0	18,936	0.00	336	3.46	3.48	0.00	779,766.60	0.00	56.02	0.00	1.62	1.25	0.00	1.46	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT /*+ PARALLEL(32) */ RSH...
9wksv5v55p6	1,187.91	176.60	0.31	19,196.489	0	0	0.00	0	0	0	0	0	0	0	1.62	1.25	0.00	1.46	0.01	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT CCB.COUNTER_ID, CCB.CTR...
2fwr5v5v55p6	1,177.59	190.66	970.49	9,889,603	1,778.473	0	96.94	0	0	0	0	0	0	0	1.26	0.26	0.07	0.05	1.68	2.06	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	SELECT "AT" "ACTUAL_EFFORT", ...

Segment Statistics

The Segment Statistics section provides a list of the most active segments. You can review this list to determine if any of the segments could benefit from a reorg or purpose of data, etc.

Statistic	Owner	Tablespace Name	Object Name	Subobject Name	Obj. Type	Obj#	Dataobj#	Value	%Total	%Capture
buffer busy waits	APPLSYS	APPS_TS_TX_DATA	SYS_IOT_TOP_48394584		INDEX	48394585	48394585	4,185		40.45
	APPLSYS	APPS_TS_TX_IDX	WF_ITEM_ATTRIBUTE_VALUES_PK		INDEX	35166	38045114	2,724		26.33
	APPLSYS	APPS_TS_TX_DATA	FND_CONCURRENT_REQUESTS		TABLE	34562	258437	1,363		13.17
	APPLSYS	APPS_TS_TX_DATA	SYS_IOT_TOP_2903659		INDEX	2903661	58983384	1,099		10.62
	APPLSYS	APPS_TS_TX_DATA	WF_ITEM_ATTRIBUTE_VALUES		TABLE	35160	38044429	976		9.43
db block changes	APPS	APPS_TS_TX_DATA	WF_ITEM_ATTRIBUTE_VALUES_ISO_V		TABLE	1195754	1195754	100,330,112		67.22
	CSI	APPS_TS_TX_DATA	CSI_COUNTER_READINGS		TABLE	395813	395813	26,957,648		18.06
	XXTGC	XXTGC_DATA			INDEX	36569101	36569101	19,530,992		13.09
	XXTGC	XXTGC_DATA			INDEX	36569099	36569099	1,253,488		0.84
	APPLSYS	APPS_TS_TX_IDX	WF_ITEM_ATTRIBUTE_VALUES_PK		INDEX	35166	38045114	1,180,944		0.79
gc buffer busy	CSI	APPS_TS_TX_IDX			INDEX	395820	395820	554,138		72.21
	CSI	APPS_TS_TX_DATA	IDXS\$S1C110001		INDEX	35049508	35049508	136,714		17.81
	CSI	APPS_TS_TX_DATA	CSI_COUNTER_READINGS		TABLE	395679	395679	29,450		3.84
	APPLSYS	APPS_TS_TX_DATA	SYS_IOT_TOP_2903659		INDEX	2903661	58983384	28,549		3.72
	APPLSYS	APPS_TS_TX_DATA	SYS_IOT_TOP_48394584		INDEX	48394585	48394585	18,570		2.42
gc cr blocks received	OKS	APPS_TS_TX_DATA	OKS_BCL_PR		TABLE	170107	269910	311,272	14.52	37.91
	APPS	APPS_TS_TX_DATA			TABLE	1195754	1195754	266,189	12.42	32.42
	CSI	APPS_TS_TX_DATA	CSI_COUNTER_READINGS		TABLE	395813	395813	162,485	7.58	19.79
	APPLSYS	APPS_TS_TX_DATA	FND_CONCURRENT_REQUESTS		TABLE	34562	258437	42,664	1.99	5.20
	CS	APPS_TS_TX_DATA	CS_INCIDENTS_ALL_B		TABLE	31966	264755	38,465	1.79	4.68
gc cr blocks served	OKS	APPS_TS_TX_DATA			TABLE	170107	269910	311,272	14.52	37.91
	APPS	APPS TS TX DATA			TABLE	1195754	1195754	266,189	12.42	32.42

Active Session History (ASH)

The ASH section provides information about active sessions and other relevant information. The following information can be found within the ASH sections:

SQL With Top Events & Row Sources

This section lists information about the relevant SQLs executing full table scans and waits events.

Top SQL with Top Events

- Top SQL statements by DB Time along with the top events by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Event is the percentage of DB Time due to the event that the SQL is waiting on.
- % Row Source is the percentage of DB Time due to the row source for the SQL waiting on the event.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Event	% Event	Top Row Source	% Row Source	SQL Text
9wkv6vj4c5t5p	907426988	186	3.84	CPU + Wait for CPU	3.81	INDEX - SKIP SCAN	0.46	SELECT ATTR_NO, ATTR_NAME, ATT...
fn55v8hd46s38	2134853384	5	1.90	CPU + Wait for CPU	1.51	INDEX - SKIP SCAN	1.14	SELECT HTA.ATTRIBUTE1, HTA.AT...
061fpuft1by2v9	1703691936	13	1.34	CPU + Wait for CPU	0.77	HASH JOIN	0.22	SELECT CCB.COUNTER_ID, CCB.CTR...
0byvqim6n8n3k	1341434170	117	1.21	CPU + Wait for CPU	1.21	TABLE ACCESS - FULL	1.21	UPDATE X...
1uttrvghv22svn	1608993741	1	1.21	CPU + Wait for CPU	1.17	TABLE ACCESS - FULL	1.15	SELECT CASE WHEN SUB_ACCOUNT...

[Back to Active Session History\(ASH\) Report](#)
[Back to Top](#)

Top SQL with Top Row Sources

- Top SQL statements by DB Time along with the top row sources by DB Time for those SQLs.
- % Activity is the percentage of DB Time due to the SQL.
- % Row Source is the percentage of DB Time spent on the row source by that SQL.
- % Event is the percentage of DB Time spent on the event by the SQL executing the row source.
- Executions is the number of executions of the SQL that were sampled in ASH.

SQL ID	Plan Hash	Executions	% Activity	Row Source	% Row Source	Top Event	% Event	SQL Text
9wkv6vj4c5t5p	907426988	186	3.84	INDEX - SKIP SCAN	0.46	CPU + Wait for CPU	0.46	SELECT ATTR_NO, ATTR_NAME, ATT...
fn55v8hd46s38	2134853384	5	1.90	INDEX - SKIP SCAN	1.15	CPU + Wait for CPU	1.14	SELECT HTA.ATTRIBUTE1, HTA.AT...
061fpuft1by2v9	1703691936	13	1.34	INDEX - FULL SCAN	0.57	gc current block 2-way	0.24	SELECT CCB.COUNTER_ID, CCB.CTR...
0byvqim6n8n3k	1341434170	117	1.21	TABLE ACCESS - FULL	1.21	CPU + Wait for CPU	1.21	UPDATE X...
1uttrvghv22svn	1608993741	1	1.21	TABLE ACCESS - FULL	1.15	CPU + Wait for CPU	1.15	SELECT CASE WHEN SUB_ACCOUNT...

[Back to Active Session History\(ASH\) Report](#)
[Back to Top](#)

Top Sessions

This section shows which user sessions consume the most resources.

Top Sessions

- '# Samples Active' shows the number of ASH samples in which the session was found waiting for that particular event. The percentage shown in this column is calculated with respect to wall clock time and not total database activity.
- 'XIDs' shows the number of distinct transaction IDs sampled in ASH when the session was waiting for that particular event

Sid, Serial#	% Activity	Event	% Event	User	Program	# Samples Active	XIDs
3331.23675	1.22	CPU - Wait for CPU	0.99	APPS	oracleC	96/118 [82%]	0
1495.21871	1.21	db file sequential read	0.89	APPS	fmwweb	86/118 [73%]	0
3075.22833	1.21	CPU - Wait for CPU	1.17	APPS	Standar	113/118 [96%]	0
3379.47092	1.21	db file sequential read	0.99	APPS	INCIAR	96/118 [82%]	2
3569.50627	1.21	CPU - Wait for CPU	1.19	APPS	Standar	115/118 [98%]	28

Automatic Database Diagnostic Monitor (ADDM)

ADDM is a diagnostic tool that analyzes the data in the Automatic Workload Repository (AWR) and provides recommendations for resolving issues found. ADDM often suggests utilizing Tuning Advisor to improve the execution of certain SQL. The recommended next steps would be to execute SQL tuning advisors on the identified SQL and review the various recommendations provided.

Findings and Recommendations

Finding 1: Top SQL Statements

Impact is 6 active sessions, 65.48% of total activity.

SQL statements consuming significant database time were found. These statements offer a good opportunity for performance improvement.

Recommendation 1: SQL Tuning

Estimated benefit is 2.14 active sessions, 23.37% of total activity.

Action

Run SQL Tuning Advisor on the SELECT statement with SQL_ID "afy1b42h6ksju".

Related Object

SQL statement with SQL_ID afy1b42h6ksju.

Benefits of AWR

- **Performance Analysis:** DBAs can utilize AWR reports to identify performance bottlenecks, analyze query execution plans, and understand overall database workload characteristics.
- **Capacity Planning:** AWR data helps in capacity planning by providing insights into resource utilization trends and future growth projections.

- **Problem Diagnosis:** When performance issues arise, AWR reports become crucial for pinpointing the root cause, allowing for targeted troubleshooting.
- **Self-Tuning:** Oracle Database can leverage AWR data for self-tuning purposes, potentially adjusting configuration parameters to optimize performance.
- **Baseline Comparisons:** By comparing historical AWR snapshots, DBAs can assess the impact of configuration changes or workload shifts on performance.

Database Statistic Levels

- Database statistics provide information about the type of database load and the resources being used by the database. To effectively measure database performance, statistics must be available.
- Some of the more important database statistics that are used to measure the performance of Oracle Database:
 - Time Model Statistics
 - Active Session History Statistics
 - Wait Events Statistics
 - Session and System Statistics

Time Model Statistics

- Time model statistics use time to identify quantitative effects about specific actions performed on the database, such as logon operations and parsing. The most important time model statistic is database time, or DB time. This statistic represents the total time spent in database calls for foreground sessions and is an indicator of the total instance workload. DB time is measured cumulatively from the time of instance startup and is calculated by aggregating the CPU and wait times of all foreground sessions not waiting on idle wait events (non-idle user sessions).
- When tuning an Oracle database, each component has its own set of statistics. To look at the system as a whole, it is necessary to have a common scale for comparisons. Many Oracle Database advisors and reports thus describe statistics in terms of time.
- Ultimately, the objective in tuning an Oracle database is to reduce the time that users spend in performing an action on the database, or to

simply reduce DB time. Time model statistics are accessible from the V\$SESS_TIME_MODEL and V\$SYS_TIME_MODEL views.

Active Session History Statistics

- Any session that is connected to the database and is waiting for an event that does not belong to the Idle wait class is considered an active session. Oracle Database samples active sessions every second and stores the sampled data in a circular buffer in the shared global area (SGA).
- The sampled session activity is accessible using the V\$ACTIVE_SESSION_HISTORY view. Each session sample contains a set of rows and the V\$ACTIVE_SESSION_HISTORY view returns one row for each active session per sample, starting with the latest session sample rows. Because the active session samples are stored in a circular buffer in the SGA, the greater the system activity, the smaller the number of seconds of session activity that can be stored. This means that the duration for which a session sample is displayed in the V\$ view is completely dependent on the level of database activity. Because the content of the V\$ view can become quite large during heavy system activity, only a portion of the session samples is written to disk.
- By capturing only active sessions, a manageable set of data can be captured with its size being directly related to the work being performed, rather than the number of sessions allowed on the system. Active Session History (ASH) enables you to examine and perform detailed analysis on both current data in the V\$ACTIVE_SESSION_HISTORY view and historical data in the DBA_HIST_ACTIVE_SESS_HISTORY view, often avoiding the need to replay the workload to trace additional performance information. ASH also contains execution plan information for each captured SQL statement. You can use this information to identify which part of SQL execution contributed most to the SQL elapsed time. The data present in ASH can be rolled up in various dimensions that it captures, including:

- SQL identifier of SQL statement
 - SQL plan identifier and hash value of the SQL plan used to execute the SQL statement
 - SQL execution plan information
 - Object number, file number, and block number
 - Wait event identifier and parameters
 - Session identifier and session serial number
 - Module and action name
 - Client identifier of the session
 - Service hash identifier
 - Consumer group identifier
- You can gather this information over a specified duration into an ASH report.
 - Active session history sampling is also available for Active Data Guard physical standby instances and Oracle Automatic Storage Management (Oracle ASM) instances. On these instances, the current session activity is collected and displayed in the V\$ACTIVE_SESSION_HISTORY view, but not written to disk.

Session and System Statistics

- A large number of cumulative database statistics on a system and session level are accessible using the V\$SYSSTAT and V\$SESSTAT views.

STATISTICS_LEVEL

- **STATISTICS_LEVEL** specifies the level of collection for database and operating system statistics. The Oracle Database collects these statistics for a variety of purposes, including making self-management decisions.

Property	Description
Parameter type	String
Syntax	STATISTICS_LEVEL = { ALL TYPICAL BASIC }
Default value	TYPICAL
Modifiable	ALTER SESSION, ALTER SYSTEM
Modifiable in a PDB	Yes
Basic	No

- The default setting of **TYPICAL** ensures collection of all major statistics required for database self-management functionality and provides best overall performance. The default value should be adequate for most environments.
- When the **STATISTICS_LEVEL** parameter is set to **ALL**, additional statistics are added to the set of statistics collected with the **TYPICAL** setting. The additional statistics are timed operating system statistics and plan execution statistics.
- Setting the **STATISTICS_LEVEL** parameter to **BASIC** disables the collection of many of the important statistics required by Oracle Database features and functionality, including:
 - Automatic Workload Repository (AWR) Snapshots
 - Automatic Database Diagnostic Monitor (ADDM)
 - All server-generated alerts
 - Automatic SGA Memory Management
 - Automatic optimizer statistics collection
 - Object level statistics
 - End to End Application Tracing (V\$CLIENT_STATS)
 - Database time distribution statistics (V\$SESS_TIME_MODEL and V\$SYS_TIME_MODEL)
 - Service level statistics
 - Buffer cache advisory
 - MTTR advisory

- Shared pool sizing advisory
- Segment level statistics
- PGA Target advisory
- Timed statistics
- Monitoring of statistics

Note:

Oracle strongly recommends that you do not disable these important features and functionality.

- When the STATISTICS_LEVEL parameter is modified by ALTER SYSTEM, all advisories or statistics are dynamically turned on or off, depending on the new value of STATISTICS_LEVEL. When modified by ALTER SESSION, the following advisories or statistics are turned on or off in the local session only. Their systemwide state is not changed:
 - Timed statistics
 - Timed operating system statistics
 - Plan execution statistics
- The V\$STATISTICS_LEVEL view displays information about the status of the statistics or advisories controlled by the STATISTICS_LEVEL parameter.
- The STATISTICS_LEVEL parameter in Oracle Database controls the collection of various database and operating system statistics. These statistics are crucial for the database to function efficiently and make self-management decisions.

Settings and Impact:

BASIC: This setting disables the collection of most important statistics. While it reduces resource consumption, it also hinders the database's ability to self-manage and optimize performance. Not recommended for most environments.

TYPICAL (Default): This is the default setting and offers a good balance. It collects essential statistics for workload analysis, wait event monitoring, and basic self-management features like buffer cache management.

ALL: This setting enables the collection of all available statistics, including timed OS statistics and plan execution statistics. Provides the most

comprehensive data for in-depth performance analysis and advanced self-management features. However, it can consume more resources.

Managing the Automatic Workload Repository

Following are the activities to perform for managing AWR

- [Enabling the Automatic Workload Repository](#)
- [Managing Snapshots](#)
- [Managing Baselines](#)
- [Managing Baseline Templates](#)
- [Transporting Automatic Workload Repository Data to Another System](#)
- [Using Automatic Workload Repository Views](#)
- [Managing Automatic Workload Repository in a Multitenant Environment](#)
- [Managing Automatic Workload Repository in Active Data Guard Standby Databases](#)

Enabling the Automatic Workload Repository

Gathering database statistics using AWR is enabled by default and is controlled by the STATISTICS_LEVEL initialization parameter.

To enable statistics gathering by AWR:

- Set the STATISTICS_LEVEL parameter to TYPICAL or ALL.

The default setting for this parameter is TYPICAL.

Setting STATISTICS_LEVEL to BASIC disables many Oracle Database features, including AWR, and is not recommended. If STATISTICS_LEVEL is set to BASIC, you can still manually capture AWR statistics using the DBMS_WORKLOAD_REPOSITORY package.

However, because in-memory collection of many system statistics—such as segments statistics and memory advisor information—will be disabled, the statistics captured in these snapshots may not be complete.

Overview of the Automatic Database Diagnostic Monitor

- The Automatic Workload Repository (AWR) stores performance related statistics for an Oracle database. The Automatic Database Diagnostic Monitor (ADDM) is a diagnostic tool that analyzes the AWR data on a regular basis, locates root causes of any performance problems, provides recommendations for correcting the problems, and identifies non-problem areas of the system. Because AWR is a repository of historical performance data, ADDM can analyze performance issues after the event, often saving time and resources in reproducing a problem.
- In most cases, ADDM output should be the first place that a DBA looks when notified of a performance problem. ADDM provides the following benefits:
 - Automatic performance diagnostic report every hour by default
 - Problem diagnosis based on decades of tuning expertise
 - Time-based quantification of problem impacts and recommendation benefits
 - Identification of root cause, not symptoms
 - Recommendations for treating the root causes of problems
 - Identification of non-problem areas of the system
 - Minimal overhead to the system during the diagnostic process
- Tuning is an iterative process, and fixing one problem can cause the bottleneck to shift to another part of the system. Even with the benefit of ADDM analysis, it can take multiple tuning cycles to reach acceptable system performance. ADDM benefits apply beyond production systems; on development and test systems, ADDM can provide an early warning of performance issues.

ADDM Analysis

- An ADDM analysis can be performed on a pair of AWR snapshots and a set of instances from the same database. The pair of AWR snapshots define the time period for analysis, and the set of instances define the target for analysis.
- If you are using Oracle Real Application Clusters (Oracle RAC), then ADDM has three analysis modes:
 - Database

In Database mode, ADDM analyzes all instances of the database.

- Instance

In Instance mode, ADDM analyzes a particular instance of the database.

- Partial

In Partial mode, ADDM analyzes a subset of all database instances.

- If you are not using Oracle RAC, then ADDM can only function in Instance mode because only one instance of the database exists.
- An ADDM analysis is performed each time an AWR snapshot is taken and the results are saved in the database. The time period analyzed by ADDM is defined by the last two snapshots (the last hour by default). ADDM will always analyze the specified instance in Instance mode. For non-Oracle RAC or single instance environments, the analysis performed in the Instance mode is the same as a database-wide analysis. If you are using Oracle RAC, then ADDM also analyzes the entire database.
- After ADDM completes its analysis, you can view the ADDM results using either Cloud Control, or DBMS_ADDM package subprograms, or DBA_ADDM_* and DBA_ADVISOR_* views.
- ADDM analysis is performed top down, first identifying symptoms, and then refining them to reach the root causes of performance problems. The goal of the analysis is to reduce a single throughput metric called DB time. DB time is the fundamental measure of database performance, and is the cumulative time spent by the database in processing user requests. It includes wait time and CPU time of all non-idle user foreground sessions. DB time is displayed in the V\$SESS_TIME_MODEL and V\$SYS_TIME_MODEL views.
- By reducing DB time, the database is able to support more user requests using the same resources, which increases throughput. The problems reported by ADDM are sorted by the amount of DB time they are responsible for. System areas that are not responsible for a significant portion of DB time are reported as non-problem areas.
- The types of problems that ADDM considers include the following:
 - CPU bottlenecks - Is the system CPU bound by Oracle Database or some other application?
 - Undersized Memory Structures - Are the Oracle Database memory structures, such as the SGA, PGA, and buffer cache, adequately sized?

- I/O capacity issues - Is the I/O subsystem performing as expected?
- High-load SQL statements - Are there any SQL statements which are consuming excessive system resources?
- High-load PL/SQL execution and compilation, and high-load Java usage
- Oracle RAC specific issues - What are the global cache hot blocks and objects; are there any interconnect latency issues?
- Sub-optimal use of Oracle Database by the application - Are there problems with poor connection management, excessive parsing, or application level lock contention?
- Database configuration issues - Is there evidence of incorrect sizing of log files, archiving issues, excessive checkpoints, or sub-optimal parameter settings?
- Concurrency issues - Are there buffer busy problems?
- Hot objects and top SQL for various problem areas
- ADDM also documents the non-problem areas of the system. For example, wait event classes that are not significantly impacting the performance of the system are identified and removed from the tuning consideration at an early stage, saving time and effort that would be spent on items that do not impact overall system performance

Using ADDM in a Multitenant Environment

- Starting with Oracle Database 12c, ADDM is enabled by default in the root container of a multitenant container database (CDB). Starting with Oracle Database 19c, you can also use ADDM in a pluggable database (PDB).
- In a CDB, ADDM works in the same way as it works in a non-CDB, that is, the ADDM analysis is performed each time an AWR snapshot is taken on a CDB root or a PDB, and the ADDM results are stored on the same database system where the snapshot is taken. The time period analyzed by ADDM is defined by the last two snapshots (the last hour by default).
- After ADDM completes its analysis, you can view the ADDM results using any of the following methods:
 - Using Enterprise Manager Cloud Control (Cloud Control)
 - Using the DBA_ADDM_* and DBA_ADVISOR_* views

Note:

- ADDM is enabled by default in a CDB root.
- ADDM does not work in a PDB by default, because automatic AWR snapshots are disabled by default in a PDB. To use ADDM in a PDB, you must enable automatic AWR snapshots in the PDB.
- A user whose current container is the CDB root can view ADDM results for the entire CDB. The ADDM results can include information about multiple PDBs. ADDM results related to a PDB are not included if the PDB is unplugged. The ADDM results stored on the CDB root cannot be viewed when the current container is a PDB.
- ADDM results on a PDB provide only PDB-specific findings and recommendations. A user whose current container is a PDB can view ADDM results for the current PDB only. The ADDM results exclude findings that apply to the CDB as a whole, for example, I/O problems relating to the buffer cache size.
- Enabling AWR snapshots on a PDB does not change the ADDM report on the CDB root.
- AWR data on a PDB cannot be accessed from the CDB root.

PDB-Level ADDM Restrictions

Unlike in a non-CDB, ADDM does not report the following issues in a PDB, because these issues apply to a CDB as a whole and do not apply to an individual PDB:

- I/O problems due to:
 - undersized buffer cache
 - undersized streams pool
 - excessive temporary writes
 - excessive checkpoint writes
 - excessive undo writes
 - excessive PQ checkpoint writes
 - excessive truncate writes
 - excessive tablespace DDL checkpoint
 - I/O capacity limit
- SQL hard parsing issues due to:
 - cursor aging

- out-of-memory failed parse
- SGA sizing issues

ADDM also does not report the following issues in a PDB, because these issues cannot be resolved at a PDB level:

- Cluster messaging related issues, such as network latency, congestion, contention, and lost blocks
- Log file switch waits on archiving and on checkpoint incomplete
- Too many free-buffer waits
- Contention on log buffer waits
- Waits due to CPU bottleneck
- Operating system VM paging
- Session slot wait event
- CPU quantum wait event
- RMAN related wait events, such as PQ queued wait event, PGA limit wait event, and I/O queue wait event

Enabling ADDM in a Pluggable Database

- ADDM does not work in a pluggable database (PDB) by default, because automatic AWR snapshots are disabled by default in a PDB. To use ADDM in a PDB, you must enable automatic AWR snapshots in the PDB by setting the `AWR_PDB_AUTOFLUSH_ENABLED` initialization parameter to `TRUE` and AWR snapshot interval greater than 0.

To enable ADDM in a PDB:

1. Set the `AWR_PDB_AUTOFLUSH_ENABLED` initialization parameter to `TRUE` in the PDB using the following command:

```
SQL> ALTER SYSTEM SET AWR_PDB_AUTOFLUSH_ENABLED=TRUE;
```
2. Set the AWR snapshot interval greater than 0 in the PDB using the command as shown in the following example:

```
SQL> EXEC dbms_workload_repository.modify_snapshot_settings(interval=>60);
```

Setting Up ADDM

- Automatic database diagnostic monitoring is enabled by default and is controlled by the `CONTROL_MANAGEMENT_PACK_ACCESS` and the `STATISTICS_LEVEL` initialization parameters.
- The `CONTROL_MANAGEMENT_PACK_ACCESS` parameter should be set to `DIAGNOSTIC` or `DIAGNOSTIC+TUNING` to enable automatic database diagnostic monitoring. The default setting is `DIAGNOSTIC+TUNING`. Setting `CONTROL_MANAGEMENT_PACK_ACCESS` to `NONE` disables ADDM.
- The `STATISTICS_LEVEL` parameter should be set to the `TYPICAL` or `ALL` to enable automatic database diagnostic monitoring. The default setting is `TYPICAL`. Setting `STATISTICS_LEVEL` to `BASIC` disables many Oracle Database features, including ADDM, and is strongly discouraged.
- ADDM analysis of I/O performance partially depends on a single argument, `DBIO_EXPECTED`, that describes the expected performance of the I/O subsystem. The value of `DBIO_EXPECTED` is the average time it takes to read a single database block in microseconds. Oracle Database uses the default value of 10 milliseconds, which is an appropriate value for most modern hard drives. If your hardware is significantly different—such as very old hardware or very fast RAM disks—then consider using a different value.

To determine the correct setting for the `DBIO_EXPECTED` parameter:

1. Measure the average read time of a single database block read for your hardware. Note that this measurement is for random I/O, which includes seek time if you use standard hard drives. Typical values for hard drives are between 5000 and 20000 microseconds.
2. Set the value one time for all subsequent ADDM executions.
For example, if the measured value is 8000 microseconds, you should execute the following command as SYS user:

```
EXECUTE DBMS_ADVISOR.SET_DEFAULT_TASK_PARAMETER( 'ADDM', 'DBIO_EXPECTED', 8000);
```

Diagnosing Database Performance Problems with ADDM

- To diagnose database performance problems, first review the ADDM analysis results that are automatically created each time an AWR snapshot is taken. If a different analysis is required (such as a longer analysis period, using a different DBIO_EXPECTED setting, or changing the analysis mode), you can run ADDM manually as described in this section.
- ADDM can analyze any two AWR snapshots (on the same database), as long as both snapshots are still stored in AWR (have not been purged). ADDM can only analyze instances that are started before the beginning snapshot and remain running until the ending snapshot. Additionally, ADDM will not analyze instances that experience significant errors when generating AWR snapshots. In such cases, ADDM will analyze the largest subset of instances that did not experience these problems.
- The primary interface for diagnostic monitoring is Cloud Control. Whenever possible, run ADDM using Cloud Control. If Cloud Control is unavailable, then run ADDM using the DBMS_ADDM package. To run the DBMS_ADDM APIs, the user must be granted the ADVISOR privilege.

Running ADDM in Database Mode

- For Oracle RAC configurations, you can run ADDM in Database mode to analyze all instances of the databases. For single-instance configurations, you can still run ADDM in Database mode; ADDM will behave as if running in Instance mode.
- To run ADDM in Database mode, use the DBMS_ADDM.ANALYZE_DB procedure

```
BEGIN
DBMS_ADDM.ANALYZE_DB (
    task_name          IN OUT VARCHAR2,
    begin_snapshot      IN      NUMBER,
    end_snapshot        IN      NUMBER,
    db_id               IN      NUMBER := NULL);
END;
/
```

- The task_name parameter specifies the name of the analysis task that will be created. The begin_snapshot parameter specifies the snapshot number of the beginning snapshot in the analysis period. The end_snapshot parameter specifies the snapshot number of the ending snapshot in the analysis period. The db_id parameter specifies the database identifier of the database that will be analyzed. If unspecified, this parameter defaults to the database identifier of the database to which you are currently connected.
- The following example creates an ADDM task in database analysis mode, and executes it to diagnose the performance of the entire database during the time period defined by snapshots 137 and 145:

```
VAR tname VARCHAR2(30);
BEGIN
    :tname := 'ADDM for 7PM to 9PM';
    DBMS_ADDM.ANALYZE_DB(:tname, 137, 145);
END;
/
```

Running ADDM in Instance Mode

- To analyze a particular instance of the database, you can run ADDM in Instance mode. To run ADDM in Instance mode, use the DBMS_ADDM.ANALYZE_INST procedure:

```
BEGIN
DBMS_ADDM.ANALYZE_INST (
    task_name          IN OUT VARCHAR2,
    begin_snapshot     IN      NUMBER,
    end_snapshot       IN      NUMBER,
    instance_number    IN      NUMBER := NULL,
    db_id              IN      NUMBER := NULL);
END;
/
```

- The task_name parameter specifies the name of the analysis task that will be created. The begin_snapshot parameter specifies the snapshot number of the beginning snapshot in the analysis period. The end_snapshot parameter specifies the snapshot number of the ending snapshot in the analysis period.

The `instance_number` parameter specifies the instance number of the instance that will be analyzed. If unspecified, this parameter defaults to the instance number of the instance to which you are currently connected. The `db_id` parameter specifies the database identifier of the database that will be analyzed. If unspecified, this parameter defaults to the database identifier of the database to which you are currently connected.

- The following example creates an ADDM task in instance analysis mode, and executes it to diagnose the performance of instance number 1 during the time period defined by snapshots 137 and 145:

```
VAR tname VARCHAR2(30);
BEGIN
    :tname := 'my ADDM for 7PM to 9PM';
    DBMS_ADDM.ANALYZE_INST(:tname, 137, 145, 1);
END;
/
```

Running ADDM in Partial Mode

- To analyze a subset of all database instances, you can run ADDM in Partial mode. To run ADDM in Partial mode, use the `DBMS_ADDM.ANALYZE_PARTIAL` procedure:

```
BEGIN
DBMS_ADDM.ANALYZE_PARTIAL (
    task_name          IN OUT VARCHAR2,
    instance_numbers   IN      VARCHAR2,
    begin_snapshot      IN      NUMBER,
    end_snapshot        IN      NUMBER,
    db_id               IN      NUMBER := NULL);
END;
/
```

- The `task_name` parameter specifies the name of the analysis task that will be created. The `instance_numbers` parameter specifies a comma-delimited list of instance numbers of instances that will be analyzed. The `begin_snapshot` parameter specifies the snapshot number of the beginning snapshot in the analysis period.

The `end_snapshot` parameter specifies the snapshot number of the ending snapshot in the analysis period. The `db_id` parameter specifies the database identifier of the database that will be analyzed. If unspecified, this parameter defaults to the database identifier of the database to which you are currently connected.

- The following example creates an ADDM task in partial analysis mode, and executes it to diagnose the performance of instance numbers 1, 2, and 4, during the time period defined by snapshots 137 and 145:

```
VAR tname VARCHAR2(30);
BEGIN
    :tname := 'my ADDM for 7PM to 9PM';
    DBMS_ADDM.ANALYZE_PARTIAL(:tname, '1,2,4', 137, 145);
END;
/
```

Displaying an ADDM Report

To display a text report of an executed ADDM task, use the `DBMS_ADDM.GET_REPORT` function:

```
DBMS_ADDM.GET_REPORT (task_name IN VARCHAR2 RETURN CLOB);
```

The following example displays a text report of the ADDM task specified by its task name using the `tname` variable:

```
SET LONG 1000000 PAGESIZE 0;

SELECT DBMS_ADDM.GET_REPORT(:tname) FROM DUAL;
```

Note that the return type of a report is a CLOB, formatted to fit line size of 80

ADDM Views

- Typically, you should view ADDM analysis using Cloud Control or DBMS_ADDM package subprograms.

However, you can also get ADDM information using the DBA_ADDM_* and DBA_ADVISOR_* views. This group of views includes:

- DBA_ADVISOR_FINDINGS

This view displays all the findings discovered by all advisors. Each finding is displayed with an associated finding ID, name, and type. For tasks with multiple executions, the name of each task execution associated with each finding is also listed.

- DBA_ADDM_FINDINGS

This view contains a subset of the findings displayed in the related DBA_ADVISOR_FINDINGS view. This view only displays the ADDM findings discovered by all advisors. Each ADDM finding is displayed with an associated finding ID, name, and type.

- DBA_ADVISOR_FINDING_NAMES

This view lists all finding names registered with the advisor framework.

- DBA_ADVISOR_RECOMMENDATIONS

This view displays the results of completed diagnostic tasks with recommendations for the problems identified in each execution. The recommendations should be reviewed in the order of the RANK column, as this relays the magnitude of the problem for the recommendation. The BENEFIT column displays the benefit to the system you can expect after the recommendation is performed. For tasks with multiple executions, the name of each task execution associated with each advisor task is also listed.

- DBA_ADVISOR_TASKS

This view provides basic information about existing tasks, such as the task ID, task name, and when the task was created. For tasks with multiple executions, the name and type of the last or current execution associated with each advisor task is also listed.

Monitoring an Oracle Database

- Monitoring the performance of a database and ensuring that it performs optimally is an important task for a database administrator. There are different features and functions included in Oracle Database that make it easy to monitor database health, identify performance problems, and implement any corrective actions
- Oracle Database makes it easy to monitor the health and performance of your database. It monitors the vital signs (or metrics) related to database health and performance, analyzes the workload running against the database, and automatically identifies any issues that need your attention as an administrator. The identified issues are presented as alerts and performance findings on the Database Home page. You can also configure Oracle Enterprise Manager Database Control (Database Control) to notify you of issues by e-mail.
- It is important that you *monitor the operation of your database on a regular basis*. Doing so *not only informs you of errors that have not yet come to your attention but also gives you a better understanding of the normal operation of your database*. Being familiar with normal behavior in turn *helps you recognize when something is wrong*.
- Database Monitoring includes followings
 - Monitoring Errors and Alerts
 - Monitoring Performance
 - Monitoring Quarantined Objects

Monitoring Errors and Alerts

We can monitor database errors and alerts to prevent, detect, and solve problems.

Note:

The easiest and best way to monitor the database for errors and alerts is with the Database Home page in Oracle Enterprise Manager Cloud Control (Cloud Control). See the Cloud Control online help for more information. This section provides alternate methods for monitoring, using data dictionary views, PL/SQL packages, and other command-line facilities.

Monitoring Errors and Alerts includes:

- **Monitoring Errors with Trace Files and the Alert Log**
- **Monitoring a Database with Server-Generated Alerts**

Monitoring Errors with Trace Files and the Alert Log

A trace file is a file that contains diagnostic data used to investigate problems. An alert log is a file that provides a chronological log of database messages and errors.

The trace file and alert log contain information about errors.

Each server and background process can write to an associated trace file. When an internal error is detected by a process, it dumps information about the error to its trace file. Some of the information written to a trace file is intended for the database administrator, and other information is for Oracle Support Services. Trace file information is also used to tune applications and instances.

The alert log is a chronological log of messages and errors, and includes the following items:

- All internal errors (ORA-00600), block corruption errors (ORA-01578), and deadlock errors (ORA-00060) that occur
- Administrative operations, such as some CREATE, ALTER, and DROP statements and STARTUP, SHUTDOWN, and ARCHIVELOG statements
- Messages and errors relating to the functions of shared server and dispatcher processes
- Errors occurring during the automatic refresh of a materialized view
- The values of all initialization parameters that had nondefault values at the time the database and instance start

Oracle Database uses the alert log to record these operations as an alternative to displaying the information on an operator's console (although some systems also display information on the console). If an operation is successful, a "completed" message is written in the alert log, along with a timestamp.

The alert log is maintained as both an XML-formatted file and a text-formatted file. You can view either format of the alert log with any text editor or you can use the ADRCI utility to view the XML-formatted version of the file with the XML tags stripped.

Check the alert log and trace files of an instance periodically to learn whether the background processes have encountered errors. For example, when the log writer process (LGWR) cannot write to a member of a log group, an error message indicating the nature of the problem is written to the LGWR trace file and the alert log. Such an error message means that a media or I/O problem has occurred and should be corrected immediately.

Oracle Database also writes values of initialization parameters to the alert log, in addition to other important statistics.

The alert log and all trace files for background and server processes are written to the Automatic Diagnostic Repository, the location of which is specified by the DIAGNOSTIC_DEST initialization parameter. The names of trace files are operating system specific, but each file usually includes the name of the process writing the file (such as LGWR and RECO).

Controlling the Size of an Alert Log

To **control the size of an alert log**, you must manually delete the file when you no longer need it. Otherwise the database continues to append to the file.

You can safely delete the alert log while the instance is running, although you should consider making an archived copy of it first. This archived copy could prove valuable if you should have a future problem that requires investigating the history of an instance.

To control the size of an alert log:

- Delete the alert log file.

Controlling the Size of Trace Files

You can **control the maximum size of all trace files** (excluding the alert log) using the initialization parameter `MAX_DUMP_FILE_SIZE`.

You can set this parameter in the following ways:

- A numerical value specifies the maximum size in operating system blocks. The specified value is multiplied by the block size to obtain the limit.
- A number followed by a K, M, or G suffix specifies the file size in kilobytes, megabytes, or gigabytes.
- UNLIMITED, which is the default, specifies no limit.
- Trace File Segmentation and `MAX_DUMP_FILE_SIZE`
Oracle Database can automatically segment trace files based on the limit you specify with the `MAX_DUMP_FILE_SIZE` initialization parameter. When a limit is reached, the database renames the current trace file using a sequential number, and creates an empty file with the original name.

Trace File Segmentation and `MAX_DUMP_FILE_SIZE`

The following table describes how trace files are segmented based on the `MAX_DUMP_FILE_SIZE` setting.

<code>MAX_DUMP_FILE_SIZE</code> Setting	Trace File Segmentation
UNLIMITED	Trace files are not segmented.
Larger than 15M	Trace files are segmented on a boundary that is 1/5 of the <code>MAX_DUMP_FILE_SIZE</code> setting. Trace files with sizes that are less than this boundary in size are not segmented. For example, if the <code>MAX_DUMP_FILE_SIZE</code> setting is 100M, then the boundary is 20 MB (1/5 of 100 MB).
15M or less	Trace files are not segmented.

There can be up to five segments, but the total combined size of the segments cannot exceed the `MAX_DUMP_FILE_SIZE` limit. When the combined size of all segments of the trace file exceeds the specified limit, the oldest segment after the first segment is deleted, and a new, empty segment is created. Therefore, the trace file always contains the most recent trace information. The first segment is not deleted because it might contain relevant information about the initial state of the process.

Segmentation improves space management for trace files. Specifically, segmentation enables you to manage trace files in the following ways:

- You can purge old trace files when they are no longer needed.
- You can diagnose problems with smaller trace files and isolate trace files that must be packaged for the incident packaging service (IPS).

Controlling When Oracle Database Writes to Trace Files

- Background processes always write to a trace file when appropriate.
- In the case of the ARCn background process, it is possible, through the LOG_ARCHIVE_TRACE initialization parameter, to control the amount and type of trace information that is produced.
- Other background processes do not have this flexibility.
- Trace files are written on behalf of server processes whenever critical errors occur. Additionally, setting the initialization parameter SQL_TRACE = TRUE causes the SQL trace facility to generate performance statistics for the processing of all SQL statements for an instance and write them to the Automatic Diagnostic Repository.
- Optionally, you can request that trace files be generated for server processes. Regardless of the current value of the SQL_TRACE initialization parameter, each session can enable or disable trace logging on behalf of the associated server process by using the SQL statement ALTER SESSION SET SQL_TRACE. This example enables the SQL trace facility for a specific session:

```
ALTER SESSION SET SQL_TRACE TRUE;
```

Use the DBMS_SESSION or the DBMS_MONITOR packages to control SQL tracing for a session.

Reading the Trace File for Shared Server Sessions

- If shared server is enabled, each session using a dispatcher is routed to a shared server process, and trace information is written to the server trace file only if the session has enabled tracing (or if an error is encountered). Therefore, to track tracing for a specific session that connects using a dispatcher, you might have to explore several shared server trace files.

- To help you, Oracle provides a command line utility program, trcsess, which consolidates all trace information pertaining to a user session in one place and orders the information by time.

Monitoring a Database with Server-Generated Alerts

A server-generated alert may contain suggestions for correcting the problem. Notifications are also provided when the problem condition has been cleared.

Alerts are automatically generated when a problem occurs or when data does not match expected values for metrics, such as the following:

- Physical Reads Per Second
- User Commits Per Second
- SQL Service Response Time

Server-generated alerts can be based on threshold levels or can issue simply because an event has occurred. Threshold-based alerts can be triggered at both threshold warning and critical levels. The value of these levels can be customer-defined or internal values, and some alerts have default threshold levels which you can change if appropriate. For example, by default a server-generated alert is generated for tablespace space usage when the percentage of space usage exceeds either the 85% warning or 97% critical threshold level. Examples of alerts not based on threshold levels are:

- Snapshot Too Old
- Resumable Session Suspended
- Recovery Area Space Usage

An alert message is sent to the predefined persistent queue ALERT_QUE owned by the user SYS. Cloud Control reads this queue and provides notifications about outstanding server alerts, and sometimes suggests actions for correcting the problem. The alerts are displayed on the Cloud Control Database Home page and can be configured to send email or pager notifications to selected administrators. If an alert cannot be written to the alert queue, a message about the alert is written to the Oracle Database alert log.

Background processes periodically flush the data to the Automatic Workload Repository to capture a history of metric values. The alert history table and ALERT_QUE are purged automatically by the system at regular intervals.

Setting and Retrieving Thresholds for Server-Generated Alerts

- You can view and change threshold settings for the server alert metrics using the SET_THRESHOLD and GET_THRESHOLD procedures of the DBMS_SERVER_ALERT PL/SQL package.

Note: The most convenient way to set and retrieve threshold values is to use the graphical interface of Cloud Control. See the Cloud Control online help about managing alerts for instructions.

Setting Threshold Levels

The SET_THRESHOLD procedure in the DBMS_SERVER_ALERT package can set threshold levels.

To set threshold levels:

- Run SET_THRESHOLD procedure in the DBMS_SERVER_ALERT package, and specify the appropriate arguments.
- The following example shows how to set thresholds with the SET_THRESHOLD procedure for CPU time for each user call for an instance:

```
DBMS_SERVER_ALERT.SET_THRESHOLD(
    DBMS_SERVER_ALERT.CPU_TIME_PER_CALL,
    DBMS_SERVER_ALERT.OPERATOR_GE, '8000',
    DBMS_SERVER_ALERT.OPERATOR_GE, '10000', 1, 2, 'inst1',
    DBMS_SERVER_ALERT.OBJECT_TYPE_SERVICE,
    'main.regress.rdbms.dev.us.example.com');
```

In this example, a warning alert is issued when CPU time exceeds 8000 microseconds for each user call and a critical alert is issued when CPU time exceeds 10,000 microseconds for each user call. The arguments include:

- CPU_TIME_PER_CALL specifies the metric identifier.
- The observation period is set to 1 minute. This period specifies the number of minutes that the condition must deviate from the threshold value before the alert is issued.
- The number of consecutive occurrences is set to 2. This number specifies how many times the metric value must violate the threshold values before the alert is generated.
- The name of the instance is set to `inst1`.
- The constant `DBMS_ALERT.OBJECT_TYPE_SERVICE` specifies the object type on which the threshold is set. In this example, the service name is `main.regress.rdbms.dev.us.example.com`.

Retrieving Threshold Information

The `GET_THRESHOLD` procedure in the `DBMS_SERVER_ALERT` package can retrieve threshold information.

To retrieve threshold values:

- Run the `GET_THRESHOLD` procedure in the `DBMS_SERVER_ALERT` package and specify the appropriate arguments.
- The following example retrieves threshold values:

```
DECLARE

warning_operator          BINARY_INTEGER;

warning_value             VARCHAR2(60);

critical_operator         BINARY_INTEGER;

critical_value            VARCHAR2(60);

observation_period        BINARY_INTEGER;

consecutive_occurrences  BINARY_INTEGER;

BEGIN

DBMS_SERVER_ALERT.GET_THRESHOLD(

DBMS_SERVER_ALERT.CPU_TIME_PER_CALL, warning_operator,
warning_value,
```

```

        critical_operator, critical_value,
        observation_period,

        consecutive_occurrences, 'inst1',

        DBMS_SERVER_ALERT.OBJECT_TYPE_SERVICE,
        'main.regress.rdbms.dev.us.example.com');

    DBMS_OUTPUT.PUT_LINE('Warning operator:      ' ||
        warning_operator);

    DBMS_OUTPUT.PUT_LINE('Warning value:        ' ||
        warning_value);

    DBMS_OUTPUT.PUT_LINE('Critical operator:    ' ||
        critical_operator);

    DBMS_OUTPUT.PUT_LINE('Critical value:      ' ||
        critical_value);

    DBMS_OUTPUT.PUT_LINE('Observation period:   ' ||
        observation_period);

    DBMS_OUTPUT.PUT_LINE('Consecutive occurrences:' ||
        consecutive_occurrences);

END;

/

```

You can also check specific threshold settings with the DBA_THRESHOLDS view. For example:

```

SELECT      metrics_name,      warning_value,      critical_value,
consecutive_occurrences

FROM DBA_THRESHOLDS

WHERE metrics_name LIKE '%CPU Time%';

```

Viewing Server-Generated Alerts

The easiest way to view server-generated alerts is by accessing the Database Home page of Cloud Control, but there are other methods of viewing these alerts.

If you use your own tool rather than Cloud Control to display alerts, then complete the following steps to view server-generated alerts:

1. Subscribe to the ALERT_QUEUE.
2. Read the ALERT_QUEUE.
3. Display an alert notification after setting the threshold levels for an alert

To create an agent and subscribe the agent to the ALERT_QUEUE, complete the following steps:

1. Run the CREATE_AQ_AGENT procedure of the DBMS_AQADM package.
2. Run the ADD_SUBSCRIBER procedure of the DBMS_AQADM package.
3. Associate a database user with the subscribing agent, because only a user associated with the subscribing agent can access queued messages in the secure ALERT_QUEUE.
4. Assign the enqueue privilege to the user by running the ENABLE_DB_ACCESS and GRANT_QUEUE_PRIVILEGE procedures of the DBMS_AQADM package.
5. Register with the DBMS_AQ.REGISTER procedure to receive an asynchronous notification when an alert is enqueued to ALERT_QUEUE. The notification can be in the form of email, HTTP post, or PL/SQL procedure.

To read an alert message, complete the following steps:

1. Use the DBMS_AQ.DEQUEUE procedure or OCIAQDeq call.
2. After the message has been dequeued, use the DBMS_SERVER_ALERT.EXPAND_MESSAGE procedure to expand the text of the message.

Server-Generated Alerts Data Dictionary Views

You can query data dictionary views for information about server-generated alerts.

View	Description
DBA_THRESHOLDS	Lists the threshold settings defined for the instance
DBA_OUTSTANDING_ALERTS	Describes the outstanding alerts in the database
DBA_ALERT_HISTORY	Lists a history of alerts that have been cleared
V\$ALERT_TYPES	Provides information such as group and type for each alert

<u>V\$METRICNAME</u>	Contains the names, identifiers, and other information about the system metrics
<u>V\$METRIC</u>	Contains system-level metric values
<u>V\$METRIC_HISTORY</u>	Contains a history of system-level metric values

Monitoring Performance

Monitoring performance includes monitoring locks and wait events and querying a set of data dictionary views.

Monitoring Locks

- Locks are mechanisms that prevent destructive interaction between transactions accessing the same resource. The resources can be either user objects, such as tables and rows, or system objects not visible to users, such as shared data structures in memory and data dictionary rows.
- Oracle Database automatically obtains and manages necessary locks when executing SQL statements, so you need not be concerned with such details. However, the database also lets you lock data manually.
- A deadlock can occur when two or more users are waiting for data locked by each other. Deadlocks prevent some transactions from continuing to work. Oracle Database automatically detects deadlock situations and resolves them by rolling back one of the statements involved in the deadlock, thereby releasing one set of the conflicting row locks.
- Oracle Database is designed to avoid deadlocks, and they are not common. Most often they occur when transactions explicitly override the default locking of the database. Deadlocks can affect the performance of your database, so Oracle provides some scripts and views that enable you to monitor locks.

To monitor locks:

1. Run the `catblock.sql`, which creates lock views.
2. Run the `utllockt.sql` script, which uses the views created by `catblock.sql` to display, in a tree fashion, the sessions in the system that are waiting for locks and the locks that they are waiting for.

The location of the script files is operating system dependent.

About Monitoring Wait Events

- Wait events are statistics that are incremented by a server process to indicate that it had to wait for an event to complete before being able to continue processing. A session could wait for a variety of reasons, including waiting for more input, waiting for the operating system to complete a service such as a disk write, or it could wait for a lock or latch.
- When a session is waiting for resources, it is not doing any useful work. A large number of waits is a source of concern. Wait event data reveals various symptoms of problems that might be affecting performance, such as latch contention, buffer contention, and I/O contention.

Performance Monitoring Data Dictionary Views

You can query a set of data dictionary views to monitor an Oracle Database instance.

These views are general in their scope. Other views, more specific to a process, are discussed in the section of this book where the process is described.

View	Description
<u>V\$LOCK</u>	Lists the locks currently held by Oracle Database and outstanding requests for a lock or latch
<u>DBA_BLOCKERS</u>	Displays a session if it is holding a lock on an object for which another session is waiting
<u>DBA_WAITERS</u>	Displays a session if it is waiting for a locked object
<u>DBA_DDL_LOCKS</u>	Lists all DDL locks held in the database and all outstanding requests for a DDL lock
<u>DBA_DML_LOCKS</u>	Lists all DML locks held in the database and all outstanding requests for a DML lock
<u>DBA_LOCK</u>	Lists all locks or latches held in the database and all outstanding requests for a lock or latch
<u>DBA_LOCK_INTERNAL</u>	Displays a row for each lock or latch that is being held, and one row for each outstanding request for a lock or latch
<u>V\$LOCKED_OBJECT</u>	Lists all locks acquired by every transaction on the system
<u>V\$SESSION_WAIT</u>	Lists the resources or events for which active sessions are waiting
<u>V\$SYSSTAT</u>	Contains session statistics
<u>V\$RESOURCE_LIMIT</u>	Provides information about current and maximum global resource utilization for some system resources
<u>V\$SQLAREA</u>	Contains statistics about shared SQL area and contains one row for each SQL string. Also provides statistics about

	SQL statements that are in memory, parsed, and ready for execution
<u>V\$LATCH</u>	Contains statistics for nonparent latches and summary statistics for parent latches

Monitoring Quarantined Objects

- Object quarantine enables an Oracle database to function even when there are corrupted, unrecoverable objects. The V\$QUARANTINE view contains information about quarantined objects.

About Object Quarantine

- Object quarantine isolates an object that has raised an error and monitors the object for impacts on the system.
- Some Oracle Database errors, such as ORA-00600 and ORA-07445, typically cause the process to terminate, which can cause the database to terminate. When such an error is encountered, object quarantine attempts to isolate the resource that caused the error so that the database can continue to run. The resource is isolated in memory so that it does not affect the rest of the database. The V\$QUARANTINE view stores information about the objects that are currently quarantined.
- Most database resources can raise errors that can cause a database to terminate. For example, library cache memory objects can raise such errors.
- In a multitenant environment, a multitenant container database (CDB) can, in some cases, use object quarantine to isolate and terminate a pluggable database (PDB) that has raised a serious error instead of terminating the CDB.
- A quarantined resource typically remains quarantined until the database is restarted. If a resource is quarantined for a PDB in a CDB, then the resource is quarantined until the PDB is closed and re-opened.

Viewing Quarantined Objects

- The V\$QUARANTINE view stores information about the objects that are currently quarantined.
 1. Connect to the database as an administrative user.
 2. Query the V\$QUARANTINE view.

Example: Querying the V\$QUARANTINE View

This query shows the resources that are currently quarantined.

```
COLUMN OBJECT FORMAT A10
COLUMN ADDRESS FORMAT A10
COLUMN BYTES FORMAT 999999999
COLUMN ERROR FORMAT A20
COLUMN TIMESTAMP FORMAT A20
```

```
SELECT OBJECT, ADDRESS, BYTES, ERROR, TIMESTAMP
FROM V$QUARANTINE;
```

Your output is similar to the following:

OBJECT	ADDRESS	BYTES ERROR	TIMESTAMP
session	0000000078	9528 ORA-00600: internal	16-SEP-15 01.17.42.2
	B54BC8	error code, argument 85878 PM -07:00	
		s: [12345], [], [],	
		[], [], [], [], [],	
		[], [], [], []	

This output shows the following about the quarantined resource:

- The name of the resource is "session."
- The start address of the memory region being quarantined is 0000000078B54BC8. Typically, this is the address of the resource, such as the session in this example.
- The resource is using 9528 bytes of memory in quarantine.
- The message of the error that caused the resource to be placed in quarantine is "ORA-00600 internal error code."
- The timestamp shows the date and time of the error.

Using Advisors to Optimize Database Performance

- ❖ Advisors are powerful tools for database management. They provide specific advice on how to address key management challenges, covering a wide range of areas including space, performance, and undo management. In general, advisors produce more comprehensive recommendations than alerts. This is because alert generation is

intended to be low cost and have minimal impact on performance, whereas advisors consume more resources and perform more detailed analysis. This, along with the what-if capability of some advisors, provides vital information for tuning that cannot be procured from any other source. Some advisors are run automatically.

- ❖ Advisors are provided to help you improve database performance. These advisors include Automatic Database Diagnostic Monitor (ADDM), SQL advisors, and memory advisors. For example, the SGA Advisor graphically displays the impact on performance of changing the size of the System Global Area (SGA).
- ❖ You can run a performance advisor when faced with the following situations:
 - You want to resolve a problem in a specific area, for example, to determine why a given SQL statement is consuming 50 percent of CPU time and what to do to reduce its resource consumption. You can use the SQL Tuning Advisor.
 - You are planning to add memory to your system. You can use the Memory Advisor to determine the database performance impact of increasing your SGA or PGA (Program Global Area).

You can also invoke some of the advisors from the Performance Hub page, or through recommendations from ADDM.

Following table describes the performance advisors.

Table: Performance Advisors

Advisor	Description
Automatic Database Diagnostics Monitor (ADDM)	ADDM makes it possible for Oracle Database to diagnose its own performance and determine how any identified problems can be resolved.
SQL Tuning Advisor	The SQL Tuning Advisor analyzes one or more SQL statements and makes recommendations for improving performance. This advisor is run automatically during the maintenance periods, but can also be run manually.
Memory Advisors • Memory Advisor • SGA Advisor	The Memory Advisors provide graphical analyses of total memory target settings, SGA and PGA target settings, or SGA component size settings. You use these analyses to tune database performance and for what-if planning. Depending on the current memory

• Buffer Cache Advisor • PGA Advisor	management mode, different memory advisors are available. • If Automatic Memory Management is enabled, the Memory Advisor is available. This advisor provides advice for the total memory target for the instance. • If Automatic Shared Memory Management is enabled, then the SGA Advisor and PGA Advisor are available. • If Manual Shared Memory Management and Automatic PGA Memory are enabled, then the Buffer Cache Advisor and PGA Advisor are available.
Undo Advisor	The Undo Advisor assists in correctly sizing the undo tablespace. The Undo Advisor can also be used to set the low threshold value of the undo retention period for any Oracle Flashback requirements.
Optimizer Statistics Advisor	Optimizer Statistics Advisor is built-in diagnostic software that analyzes the quality of statistics and statistics-related tasks. The advisor task runs automatically in the maintenance window, but you can also run it on demand. You can then view the advisor report. If the advisor makes recommendations, then in some cases you can run system-generated scripts to implement them.

For example, the shared pool memory advisor graphically displays the impact on performance of changing the size of this component of the SGA.

Using the SQL Tuning Advisor

- The SQL Tuning Advisor examines a given SQL statement or a set of SQL statements and provides recommendations to improve efficiency.
- You use the SQL Tuning Advisor to tune a single SQL statement or multiple SQL statements. Typically, you run the SQL Tuning Advisor in response to an ADDM performance finding that recommends its use. You can also run it periodically on the most resource-intensive SQL statements, and on a SQL workload
- Use the SQL Tuning Advisor for tuning SQL statements. Typically, you run this advisor in response to an ADDM performance finding that recommends its use. You can also start the SQL Tuning Advisor manually. One reason is to tune statements that the Automatic SQL Tuning Advisor has not considered for tuning.

— You can run the SQL Tuning Advisor against the following sources:

- Activity—The most resource-intensive SQL statements executed during the last hour that appear on the Activity tab of the Performance Hub that might have caused recent performance problems.
- Historical SQL—A SQL statement from the last day, week, or month that appears on the Activity tab of the Performance Hub when one of the historical settings is selected in the Select Time Period field. Use this option for proactive tuning of SQL statements.
- Historical SQL from ADDM—A resource-intensive SQL statement from an ADDM task that you discover when analyzing a task on the ADDM tab of the Performance Hub.
- SQL statement in SQL Tuning Advisor—A resource-intensive SQL statement that appears as a tuning task in SQL Tuning Advisor.
- SQL tuning sets (STS)—A set of SQL statements you provide. An STS can be created from SQL statements captured by AWR snapshots or from a SQL workload.

About the Automatic SQL Tuning Advisor

- The SQL Tuning Advisor runs automatically during system maintenance windows (time periods) as a maintenance task. During each automatic run, the advisor selects high-load SQL queries in the system and generates recommendations on how to tune these queries.
- The Automatic SQL Tuning Advisor can be configured to automatically implement SQL profile recommendations. A SQL profile contains additional SQL statistics that are specific to the SQL statement and enable the query optimizer to generate a significantly better execution plan at run time. If you enable automatic implementation, then the advisor creates SQL profiles for only those SQL statements where the performance improvement would be at least threefold. Other types of recommendations, such as the creation of new indexes, refreshing optimizer statistics, or restructuring SQL, can only be implemented manually. DML statements are not considered for tuning by the Automatic SQL Tuning Advisor.

- You can view a summary of the results of automatic SQL tuning, and a detailed report about recommendations made for all SQL statements that the SQL Tuning Advisor has processed. You can then implement selected recommendations. You can also view the recommendations that were automatically implemented.
- You can disable the Automatic SQL Tuning Advisor, if desired.

Configuring the Automatic SQL Tuning Advisor

The following are some configuration tasks that you might want to perform for the Automatic SQL Tuning Advisor:

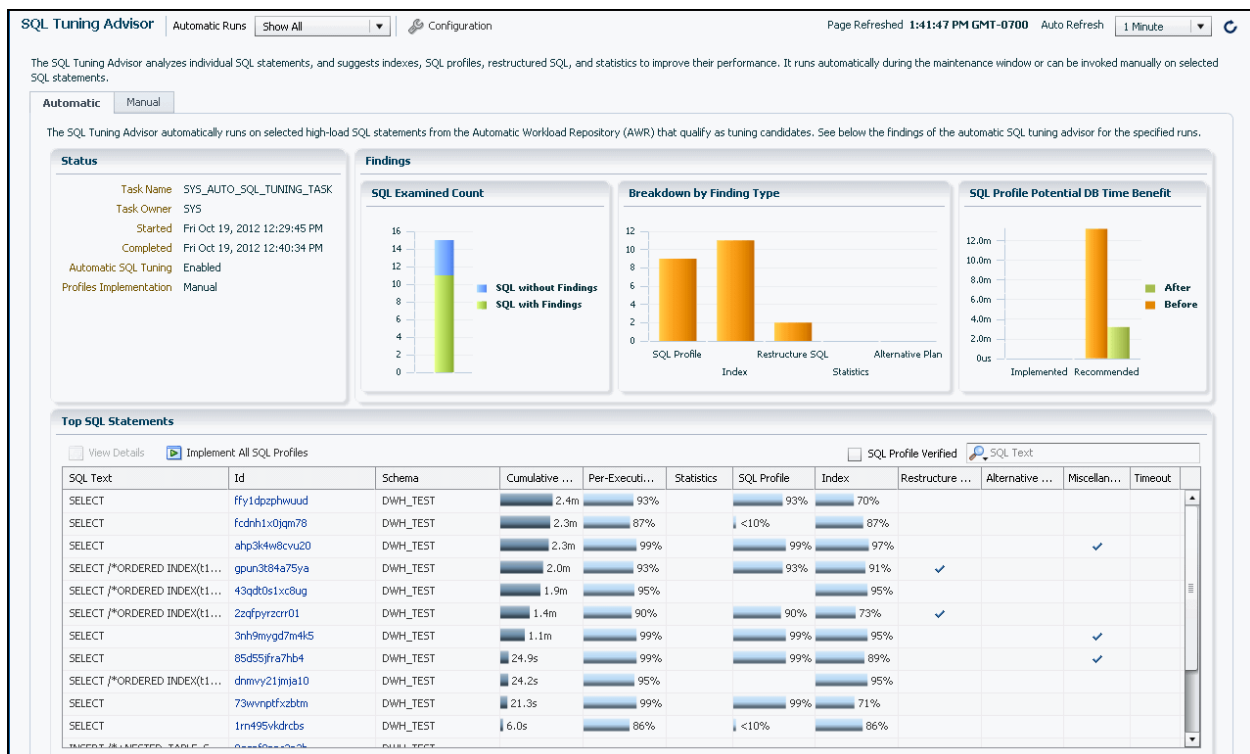
- Enable automatic implementation of SQL profile recommendations.
- Automatic implementation is disabled by default.
- Change the maximum number of SQL profiles implemented during one run of the SQL Tuning Advisor
- When automatic implementation of SQL profile recommendations is enabled, 20 SQL profiles are implemented during a run of the SQL Tuning Advisor, by default.
- Change the maximum number of SQL profiles that can be implemented overall.
- When automatic implementation of SQL profile recommendations is enabled, a total of 10000 SQL profiles can be implemented by SQL Tuning Advisor, by default.

To configure the Automatic SQL Tuning Advisor:

1. Log into EM Express as a user who has the EM_EXPRESS_ALL role.
2. In EM Express, from the **Performance** menu, select **SQL Tuning Advisor**.

The SQL Tuning Advisor page appears, with the Automatic tab showing.

The SQL tuning task that appears on the Automatic tab is the SYS_AUTO_SQL_TUNING_TASK. This tuning task is created daily by the Automatic SQL Tuning Advisor. The task includes any high-load SQL queries for which the Automatic SQL Tuning Advisor has generated tuning recommendations.



- Click the **Configuration** button.

The SQL Tuning Settings dialog box appears.

The SQL Tuning Settings dialog box contains the following settings:

- Automatic SQL Tuning:** ☒
- Time Limit per Statement (seconds):** 1200
- Automatic Implementation of SQL Profiles:** ☐
- Maximum SQL Profiles Implemented Per Execution:** 20
- Maximum SQL Profiles Implemented (Overall):** 10000

Buttons at the bottom include **Show SQL**, **OK**, and **Cancel**.

- (Optional) To disable the Automatic SQL Tuning Advisor, remove the check mark for the the **Automatic SQL Tuning** option.
Note: If you disable the Automatic SQL Tuning option, none of the other options in the SQL Tuning Settings dialog box have any effect.

5. (Optional) In the **Time Limit per Statement (seconds)** field, enter the maximum time that SQL Tuning Advisor should take to tune any single SQL statement (in seconds).
6. (Optional) In the **Automatic Implementation of SQL Profiles** field, enter a check mark to enable the automatic implementation of SQL profiles, or remove the check mark to disable the automatic implementation of SQL profiles.

Note:

*If you disable the **Automatic Implementation of SQL Profiles** option, the settings for the **Maximum SQL Profiles Implemented per Execution** and **Maximum SQL Profiles Implemented (Overall)** options have no effect.*

7. (Optional) In the **Maximum SQL Profiles Implemented per Execution** field, enter the total number of SQL profiles that can be implemented during a single daily run of the SQL Tuning Advisor. The default value is 20.
8. (Optional) In the **Maximum SQL Profiles Implemented (Overall)** field, enter the total number of SQL profiles that can be implemented overall. The default value is 10000.
9. Click **OK**.

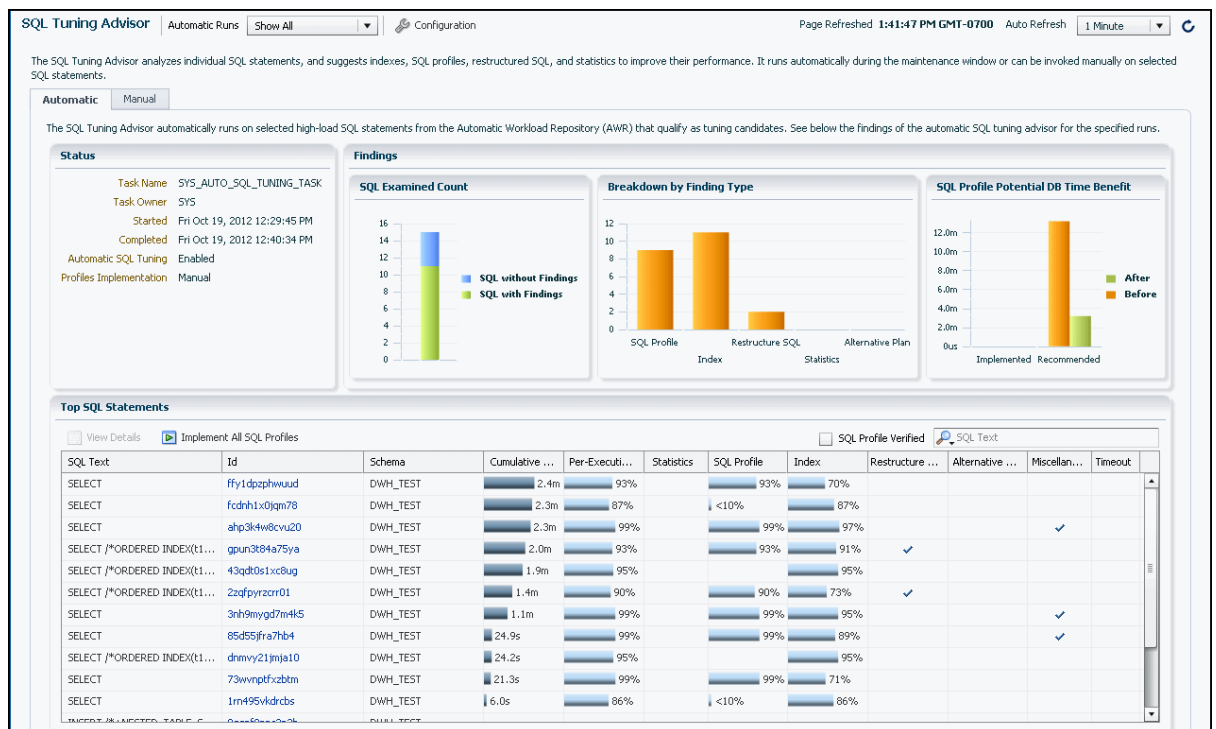
A confirmation page appears.

Viewing Automatic SQL Tuning Results

You can track the activities of the Automatic SQL Tuning Advisor with EM Express.

To view automatic SQL tuning results:

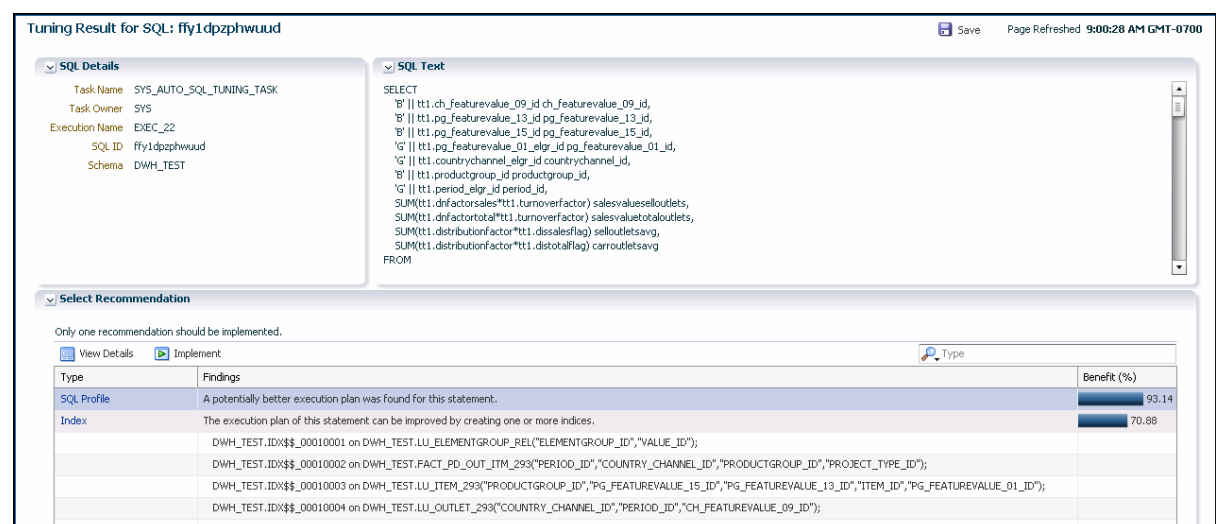
1. In EM Express, from the **Performance** menu, choose **SQL Tuning Advisor**. The SQL Tuning Advisor page appears, with the Automatic tab showing. The SQL tuning task that appears on the Automatic tab is the SYS_AUTO_SQL_TUNING_TASK. This tuning task is created daily by the Automatic SQL Tuning Advisor. The task includes any high-load SQL queries for which the Automatic SQL Tuning Advisor has generated tuning recommendations.



- In the Top SQL Statements table, select a row that includes a SQL statement for which you want to view tuning recommendations, and then click **View Details**.

In this example, the SELECT statement with a SQL ID of ffy1dpzphwuud was selected and **View Details** was clicked.

The Tuning Result for SQL page appears, which shows a summary of the tuning recommendations for the selected SQL statement.



3. The Select Recommendation section advises that only one recommendation on the page should be implemented.

To implement a recommendation, select it in the table and click **Implement**. You will be prompted to provide the necessary information to implement the recommendation.

To help you decide which (if any) of the recommendations to implement, you may want to view more details about each of the recommendations.

To view more details about a recommendation, select it in the table, then click **View Details**. In this example, the SQL Profile recommendation is selected.

The Recommendation Details page appears.

4. The top section of this page describes the performance recommendation, and the section is named after the type of recommendation. Some possible names for this section are "Stale or Missing Statistics," "Restructure SQL," and "SQL Profile." This section provides an overview of the recommendation.

Recommendation Details

Implement

Page Refreshed: 9:17:32 AM GMT-0700

SQL Profile

A SQL profile contains corrections for poor optimizer estimates discovered during SQL Tuning. This information can improve optimizer cardinality and selectivity estimates, which in turn leads the optimizer to select better plans.

A SQL profile is specific to a SQL statement.

Recommendation(s):

- A potentially better execution plan was found for this statement. Consider accepting the recommended SQL profile.

Compare Explain Plans

Original Plan

Original Plan with Adjusted Cost

Plan Using SQL Profile

Graphical

Tabular

Operation	Object	Predicate	Pruning	Operation Cost	Estimated Rows	Estimated Bytes
SELECT STATEMENT					1	111
HASH GROUP BY					1	111
VIEW					1	111
HASH GROUP BY					1	118
NESTED LOOPS					1	118
NESTED LOOPS					1	118
VIEW	VW_GBC_22				1	107
HASH GROUP BY					1	130
HASH JOIN		*			1	130
NESTED LOOPS					1	104
NESTED LOOPS				73	1	46
MERGE JOIN CARTESIAN					1	22
TABLE ACCESS BY INDEX ROWID	LU_ELEMENTGROUP_REL			1	1	11
INDEX RANGE SCAN	LU_ELEMENTGROUP_REL_IDX1			1	8	
BUFFER SORT					39	429
INLIST ITERATOR						
TABLE ACCESS BY INDEX ROWID BATCHED	LU_ELEMENTGROUP_REL			2	39	429
INDEX RANGE SCAN	LU_ELEMENTGROUP_REL_IDX1			5	39	
TABLE ACCESS BY INDEX ROWID BATCHED	LU_ELEMENTGROUP_REL					

The Compare Explain Plans section at the bottom of the page includes tabs that enable you to view one or more execution plans for the selected statement. The four tabs that can appear are the Original Plan, Original Plan with Adjusted Cost, Plan Using SQL Profile, and Alternative Plan tabs. The Graphical and Tabular buttons enable you to display an execution plan in graphical or tabular format. In this example, the execution plan is displayed in tabular format.

For recommendations that do not include a potentially better execution plan, only the Original Plan tab appears, and the operations for the original plan are shown on the tab.

When you click the Original Plan with Adjusted Cost tab, the execution plan steps are the same as the Original Plan steps, but the Original Plan with Adjusted Cost steps have different costs for the steps (as shown in the **Operation Cost** column). If you click the Plan Using SQL Profile tab, the steps are different than the Original Plan steps, and the steps have different costs (as shown in the **Operation Cost** column).

The Alternative Plan button appears when the execution history for the original plan cannot be found. In this case, if you know that the alternative plan suggested by SQL Tuning Advisor is better than the original plan, you can create a SQL plan baseline for the alternative plan so that the Oracle optimizer will pick the alternative plan for the statement in the future.

Click the **Implement** button at the top of the Recommendation Details page to implement a recommendation.

Running the SQL Tuning Advisor

Use the SQL Tuning Advisor for tuning SQL statements. Typically, you run this advisor in response to an ADDM performance finding that recommends its use. You can also start the SQL Tuning Advisor manually. One reason is to tune statements that the Automatic SQL Tuning Advisor has not considered for tuning.

As described in "About the SQL Tuning Advisor," the SQL Tuning Advisor can select SQL statements to tune from several sources. The following scenario assumes that you want to tune the SQL statements with the most activity:

To run the SQL Tuning Advisor:

1. In EM Express, from the **Performance** menu, choose **Performance Hub**.

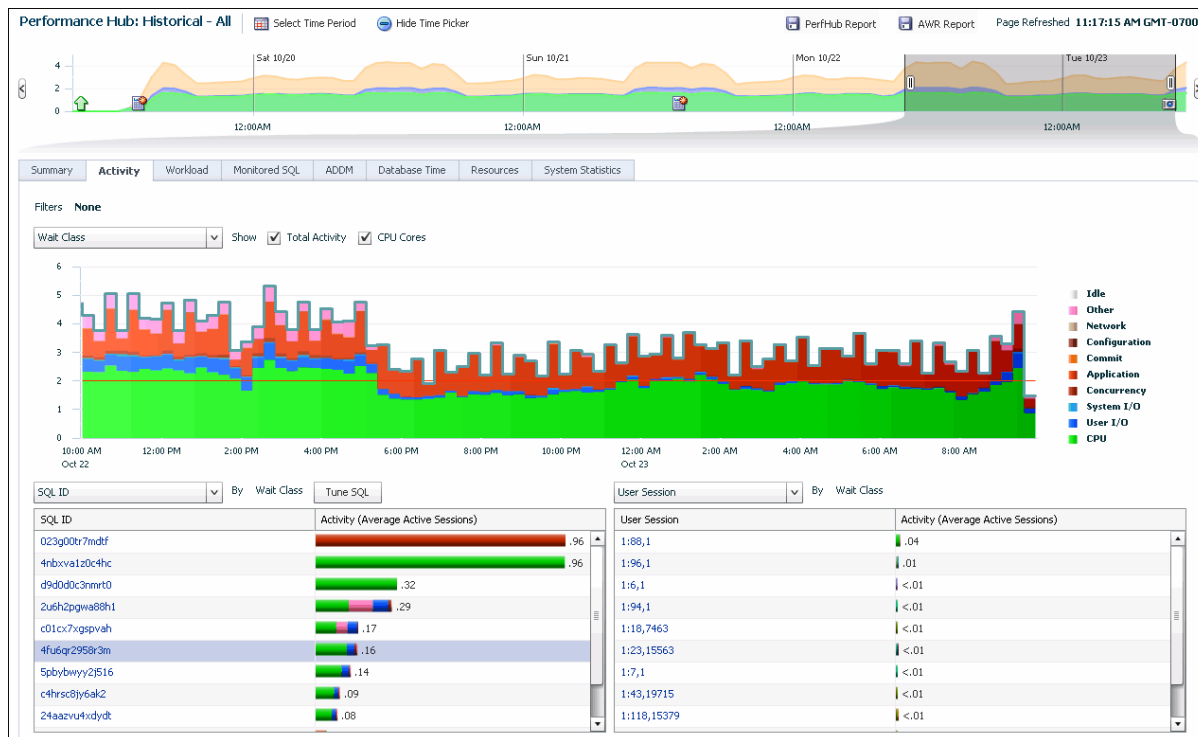
The Performance Hub page appears.

2. In the **Select Time Period** field, select the desired time period.
In this example, **Historical - All** has been selected in the **Select Time Period** field.
3. Select **Activity**.

The Activity tab appears.

- In the table at the bottom of the Activity tab, select the row that includes the SQL statement that you want to tune, and then click the **Tune SQL** button.

In this example, the SQL statement in the sixth row of the table is selected.



- The Schedule SQL Tuning Advisor wizard appears, with the Task Information page showing.

The screenshot shows the 'Tune SQL' wizard, Task Information page. The Name field is populated with 'SQLTUNE_4fu6qr2958r3m_25622306'. The Description field is empty. The OK, Cancel, and Next buttons are visible at the bottom.

On the Task Information page, you can accept the tuning task name generated by the system, or enter a name of your choosing for the tuning task that will be created for the selected SQL statement. You also have the option of entering a description for the tuning task.

Click the right arrow button.

- The Scope page appears. Specify the total time SQL Tuning Advisor should spend analyzing the statement (the default value is Unlimited), and the scope of the analysis (Comprehensive or Limited).

The screenshot shows the 'Schedule SQL Tuning Advisor' dialog box with the 'Scope' tab selected. At the top, there is a progress bar with three steps: 'Task Information', 'Scope' (highlighted), and 'Schedule'. Below the progress bar, the 'Total Time Limit (minutes)' is set to '30' in a text box. A note below the text box states 'The default value is unlimited.' Under 'Scope of Analysis', the 'Comprehensive' radio button is selected, and the 'Limited' radio button is unselected. At the bottom, there are three buttons: a left arrow, an 'OK' button with a green checkmark, and a 'Cancel' button with a red X. A right arrow button is also present on the far right.

Click the right arrow button.

- The Schedule page appears. On this page, you can schedule SQL Tuning Advisor to analyze the statement immediately or later. If you choose to have SQL Tuning Advisor analyze the statement later, specify the time that you want SQL Tuning Advisor to analyze the statement.

The screenshot shows the 'Schedule SQL Tuning Advisor' dialog box with the 'Schedule' tab selected. The progress bar at the top now highlights the 'Schedule' step. Under 'Schedule', the 'Immediately' radio button is selected, and the 'Later' radio button is unselected. Below this, the 'Time' is set to '06/06/2012' with a calendar icon, followed by time pickers for '1' hour, '41' minutes, and '0' seconds. There are also radio buttons for 'AM' and 'PM', with 'PM' selected. At the bottom, there are three buttons: a left arrow, an 'OK' button with a green checkmark, and a 'Cancel' button with a red X. A right arrow button is also present on the far right.

Click **OK** to begin the tuning task creation by SQL Tuning Advisor for the selected SQL statement.

- When the SQL Tuning Advisor finishes analyzing the SQL statement, the **Completed** (check mark) icon appears in the **Status** column on the SQL Tuning Advisor page.

Select the row that includes the SQL statement that SQL Tuning Advisor has finished analyzing, and then click **View Result** to see the recommendations the SQL Tuning Advisor has for this SQL statement. In this example, the first row is selected.

SQL Tuning Advisor

Automatic Runs

Show All

Configuration

Page Refreshed 11:07:49 AM GMT-0700

Auto Refresh

1 Minute

The SQL Tuning Advisor analyzes individual SQL statements, and suggests indexes, SQL profiles, restructured SQL, and statistics to improve their performance. It runs automatically during the maintenance window or can be invoked manually on selected SQL statements.

Automatic

Manual

This table shows a list of manually created SQL tuning tasks. You can create new SQL tuning tasks from the Performance Hub by running the SQL Tuning advisor on high-load SQL statements.

Actions

View Result

Drop

Name

Status	Name	Description	User	Start Time	Duration	Expires In (days)
	SQLTUNE_4fu5qr2958r3m_25621342		SYSTEM	Tue Oct 23, 2012 11:01:38 AM	5.2m	30
	sts_PROFILES_ALTERNATE_PLANS		SYS	Fri Oct 19, 2012 12:25:31 PM	4.1m	26
	one_sql_PROFILE_and_INDEXES		DWH_TEST	Fri Oct 19, 2012 12:25:21 PM	0us	26
	one_sql_RESTRUCTURE_SQL		DWH_TEST	Fri Oct 19, 2012 12:25:19 PM	0us	26
	one_sql_TIMEOUT_and_MORE		DWH_TEST	Fri Oct 19, 2012 12:25:13 PM	6.0s	26
	one_sql_ERROR		DWH_TEST	Fri Oct 19, 2012 12:25:12 PM	0us	26
	one_sql_INITIAL_STATUS		DWH_TEST		0us	26

The Tuning Result for SQL: SQLID page appears, which shows a summary of the tuning recommendations for the selected SQL statement.

- The Select Recommendation section at the bottom of the page shows the recommendations for tuning the SQL statement.

If there are multiple recommendations on the page, only one of them should be implemented.

To implement a recommendation, select it in the table and click **Implement**. You will be prompted to provide the necessary information to implement the recommendation.

NOTE: To help you decide which (if any) of the recommendations to implement, you may want to view more details about each of the recommendations.

*Some SQL statements may have a SQL Profile recommendation, which means that SQL Tuning Advisor recommends creating a SQL profile for the statement. When a SQL profile is recommended for a statement, the Select Recommendation section includes a recommendation with a value of **SQL Profile** in the Type column, as well as a **Validate with SPA** button. To validate the impact of implementing the SQL profile for a statement, select the SQL Profile recommendation and click **Validate with SPA**.*

To view more details about a recommendation, select it in the table, then click **View Details**.

In this example, the **Some alternative execution plans for this statement were found by searching the system's real-time and historical performance data** recommendation is selected

Tuning Result for SQL: 4fu6qr2958r3m Save Page Refreshed 11:08:42 AM GMT-0700

SQL Details

Task Name: SQTUNE_4fu6qr2958r3m_25621342

Task Owner: SYSTEM

SQL ID: 4fu6qr2958r3m

Schema: SH

SQL Text

```
select /* awrv4 */ c.channel_desc, c.channel_class,
       sum(s.quantity_sold),
       sum(s.amount_sold)
from sales s,
       channels c,
       times t
where s.channel_id = c.channel_id
and s.time_id = t.time_id
and t.fiscal_year = trunc(dbms_random.value(1998,2003))
and (ora_hash(s.rowid,5) = 1 and spin_cpu(100) > 0)
group by c.channel_desc, c.channel_class
```

Select Recommendation

Only one recommendation should be implemented.

[View Details](#) [Implement](#) Type

Type	Findings	Benefit (%)
Alternative Plan	Some alternative execution plans for this statement were found by searching the system's real-time and historical performance data.	

The Recommendation Details page appears.

10. The top section of this page describes the performance recommendation, and the section is named after the type of recommendation. Some possible names for this section are "Alternative Plan(s)," "Stale or Missing Statistics," "Restructure SQL," and "SQL Profile." This section provides an overview of the recommendation.

Recommendation Details Page Refreshed 11:13:40 AM GMT-0700

Alternative Plan(s)

While tuning a SQL statement, SQL Tuning Advisor searches real-time and historical performance data for alternative execution plans for the statement. If plans other than the original plan exist, then SQL Tuning Advisor reports an alternative plan finding.

Recommendation(s):

- Some alternative execution plans for this statement were found by searching the system's real-time and historical performance data. Because no execution history for the Original Plan was found, the SQL Tuning Advisor could not determine if any of these execution plans are superior to it. However, if you know that one alternative plan is better than the Original Plan, you can create a SQL plan baseline for it. This will instruct the Oracle optimizer to pick it over any other choices in the future.

Compare Explain Plans

[Create SQL Plan Baseline](#)

Plan Hash Value	Last Seen	Elapsed Time	Origin	Note
770275272	2012-10-23/10:08:34	19.1s	Cursor Cache	

Original Plan

☐ Graphical ☒ Tabular

Operation	Object	Predicate	Pruning	Operation Cost	Estimated Rows	Estimated Bytes
SELECT STATEMENT					5	250
PX COORDINATOR						
PX SEND QC (RANDOM)	:TQ10005				5	250
HASH GROUP BY					5	250
PX RECEIVE					5	250
PX SEND HASH	:TQ10004				5	250
HASH GROUP BY					5	250
MERGE JOIN					5	250
SORT JOIN					5	105
BUFFER SORT						
PX RECEIVE					5	105
PX SEND HYBRID HASH	:TQ10000				5	105
STATISTICS COLLECTOR						
TABLE ACCESS BY INDEX ROWID BATCHED	CHANNELS			1	5	105
INDEX FULL SCAN	CHANNELS_PK			1	5	
SORT JOIN					5	145

11. The Compare Explain Plans section at the bottom of the page includes one or more tabs that enable you to view one or more execution plans for the selected statement. The four tabs that can appear are the Original Plan, Original Plan with Adjusted Cost, Plan Using SQL Profile, and Alternative Plan tabs. The Tabular and Graphical button enable you to display an execution plan in tabular or graphical format. In this example, the execution plan is displayed in tabular format.

For recommendations that do not include a potentially better execution plan, only the Original Plan tab appears, and the operations for the original plan are shown on the Original Plan subpage.

On the Original Plan with Adjusted Cost subpage, the execution plan steps are the same as the Original Plan steps, but the Original Plan with Adjusted Cost steps have different costs for the steps (as shown in the **Operation Cost** column).

On the Plan Using SQL Profile subpage, the steps are different than the Original Plan steps, and the steps have different costs (as shown in the **Operation Cost** column).

The Alternative Plan subpage is available when the execution history for the original plan cannot be found. In this case, if you know that the alternative plan suggested by SQL Tuning Advisor is better than the original plan, you can click the **Create SQL Plan Baseline** button to create a SQL plan baseline for the alternative plan so that the Oracle optimizer will pick the alternative plan for the statement in the future.

To implement a recommendation, click the **Back** button in your browser, and implement the recommendation on the Tuning Result page.

Optimizing Memory Usage with the Memory Advisors

- Adequate physical memory has a significant impact on the performance of your Oracle Database. With its automatic memory management capabilities, Oracle Database can automatically adjust the memory distribution among the various SGA and PGA components for optimal performance. These adjustments are made within the boundaries of the total amount of memory that you allocate to the database.
- ADDM periodically evaluates the performance of your database to determine performance problems. If ADDM finds that the current amount of available memory is inadequate and adversely affecting performance, then it can recommend that you increase memory allocations. You can select new memory allocations using the Memory Advisors.
- Additionally, you can use the Memory Advisors to perform what-if analysis on the following:
 - The database performance benefits of adding physical memory to your database
 - The database performance impact of reducing the physical memory available to your database

With the Memory Advisors, you can obtain memory sizing advice as follows:

- If automatic memory management is enabled, you can get a prediction of the percentage of time saved by using a different target memory size setting for the Oracle instance.
- If automatic memory management is disabled and automatic shared memory management is enabled, you can get a prediction of the percentage of time saved by using a different total SGA size.
- If only manual shared memory management is enabled, then you can get a prediction of the percentage of reads saved by using a different database cache size.

Set Up Notification Rules

- Notification rules in Oracle Database monitoring allow you to receive alerts when specific events or conditions occur within your database.
- Setting up notification rules for Oracle database monitoring involves defining conditions and actions to be taken when certain events occur.

Here's a step-by-step guide on how to set up notification rules:

1. **Identify Monitoring Requirements: Determine** what events or conditions you want to monitor. This could include database performance metrics like CPU usage, memory consumption, disk space, query execution time, etc., as well as system health indicators like availability and uptime.
2. **Choose Monitoring Tools:** Select appropriate monitoring tools or software for Oracle database monitoring. There are various options available, for example Oracle Enterprise Manager, Nagios, Zabbix, etc.
3. **Access Monitoring System:** Log in to your chosen monitoring system with appropriate credentials and access privileges.
4. **Define Notification Rules:**
 - **Event/Condition:** Specify the event or condition that triggers the notification. For example, CPU usage exceeding a certain threshold, disk space running low, database connection failure, etc.
 - **Notification Action:** Decide what action to take when the event or condition occurs. Common actions include sending email alerts, SMS

notifications, triggering automated scripts or remediation workflows, logging the event to a file, etc.

- **Recipients:** Specify who should receive the notifications. This could be database administrators, system administrators, application owners, or any other relevant stakeholders.
 - **Severity Levels:** Assign severity levels to different events or conditions to prioritize notifications. For example, critical events may require immediate attention, while less severe issues can be addressed later.
5. **Configure Notification Channels:** Set up the communication channels through which notifications will be delivered. This may include configuring email servers, SMS gateways, webhook endpoints, etc.
 6. **Test Notification Rules:** Before putting the notification rules into production, test them thoroughly to ensure they work as expected. Trigger test events or conditions and verify that the notifications are generated and delivered correctly.
 7. **Deploy Notification Rules:** Once testing is successful, deploy the notification rules in your production environment.
 8. **Monitor and Fine-tune:** Continuously monitor the effectiveness of your notification rules and make adjustments as needed. This may involve tweaking threshold values, adding new rules for additional monitoring requirements, or optimizing notification delivery mechanisms.

By following these steps, you can effectively set up notification rules for Oracle database monitoring to proactively identify and address issues before they impact your system's performance and availability.

Performance management

- Performance management in Oracle involves a systematic approach to optimizing the performance of Oracle databases, applications, and the underlying infrastructure to ensure efficient operation and satisfactory user experience.

Tuning Information Sources

- Tuning information sources typically involves optimizing the processes used to gather performance data, metrics, and other relevant information from the database and related components.

- In Oracle, tuning your database involves optimizing its performance to deliver the best possible results for your specific workloads. To achieve this, you need access to reliable information sources that provide insights into how your database is functioning. Here are some key information sources for tuning in Oracle:
 - *Dynamic Performance Views (V\$ Views)*: Oracle provides a rich set of dynamic performance views (prefixed with V\$) that offer real-time insights into various aspects of database performance, system statistics, and resource utilization.
 - *Automatic Workload Repository (AWR)*:
AWR is a built-in repository for collecting and storing performance statistics in Oracle databases. It captures detailed performance data at regular intervals, facilitating performance analysis, tuning, and diagnostics.

Performance Monitoring

Monitoring performance includes monitoring locks and wait events and querying a set of data dictionary views. (Explained in earlier topics of this unit)

Tuning activities

Tuning activities refer to the various tasks performed to optimize the performance of an Oracle database system. These activities are aimed at improving query response times, reducing resource usage, enhancing scalability, and ensuring overall efficiency. Some common tuning activities in Oracle include:

1. **SQL Tuning**: Analyzing and optimizing SQL queries to ensure they execute efficiently. This may involve creating or modifying indexes, rewriting SQL queries, or utilizing hints to influence the optimizer's execution plan.
2. **Indexing**: Proper indexing of tables is crucial for efficient data retrieval. Tuning involves identifying appropriate columns for indexing and creating, modifying, or dropping indexes as necessary.
3. **Partitioning**: Partitioning large tables and indexes can improve query performance by allowing data to be stored and accessed in smaller, more manageable chunks.

4. **Memory Management:** Configuring memory parameters such as buffer cache size, shared pool size, and PGA (Program Global Area) size to ensure optimal memory utilization by the Oracle instance.
5. **Storage Configuration:** Optimizing storage settings such as data file placement, disk I/O configuration, and RAID levels to minimize I/O contention and improve disk performance.
6. **Optimizer Statistics:** Maintaining accurate statistics on tables and indexes to help the query optimizer generate efficient execution plans.
7. **Database Configuration:** Configuring various database parameters such as initialization parameters, redo log settings, and undo management parameters to optimize performance.
8. **Concurrency Management:** Managing locks, latches, and transactions to prevent contention and ensure efficient concurrent access to data.
9. **Monitoring and Diagnostics:** Continuously monitoring database performance using tools like Oracle Enterprise Manager or third-party monitoring tools, and diagnosing performance issues using tools like AWR (Automatic Workload Repository) and ASH (Active Session History).
10. **Hardware and Network Optimization:** Ensuring that the underlying hardware infrastructure (servers, storage, network) is properly configured and optimized for Oracle database performance.

Performance Tuning

Performance Tuning in Oracle involves following tasks:

- *Performance Planning*
- *Instance Tuning*
- *SQL Tuning*

Performance Planning

- Performance planning in Oracle involves strategizing and implementing measures to ensure that the database system meets the performance requirements of the organization.
- Performance planning describes ways to improve Oracle performance by starting with good application design and using statistics to monitor application performance. It explains the Oracle Performance Improvement Method, as well as emergency performance techniques for dealing with performance problems.

Performance Planning includes activities that can dramatically improve system performance, such as:

- Understanding Investment Options
- Understanding Scalability
- System Architecture
- Application Design Principles
- Workload Testing, Modeling, and Implementation
- Deploying New Applications

Understanding Investment Options

- With the availability of relatively inexpensive, high-powered processors, memory, and disk drives, there is a temptation to buy more system resources to improve performance. In many situations, new CPUs, memory, or more disk drives can indeed provide an immediate performance improvement. However, any performance increases achieved by adding hardware should be considered a short-term relief to an immediate problem. If the demand and load rates on the application continue to grow, then the chance of the same problem occurring soon is likely.
- In other situations, additional hardware does not improve the system's performance at all. Poorly designed systems perform poorly no matter how much extra hardware is allocated. Before purchasing additional hardware, ensure that serialization or single threading is not occurring within the application. Long-term, it is generally more valuable to increase the efficiency of your application in terms of the number of physical resources used for each business transaction.

Understanding Scalability

- Scalability is a system's ability to process more workload, with a proportional increase in system resource usage.
- In a scalable system, if you double the workload, then the system uses twice as many system resources. This sounds obvious, but due to conflicts within the system, the resource usage might exceed twice the original workload.
- Examples of poor scalability due to resource conflicts include the following:
 - ☞ Applications requiring significant concurrency management as user populations increase
 - ☞ Increased locking activities
 - ☞ Increased data consistency workload
 - ☞ Increased operating system workload
 - ☞ Transactions requiring increases in data access as data volumes increase
 - ☞ Poor SQL and index design resulting in a higher number of logical I/Os for the same number of rows returned
 - ☞ Reduced availability, because database objects take longer to maintain
- An application is said to be **unscalable** if it exhausts a system resource to the point where no more throughput is possible when its workload is increased. Such applications result in fixed throughputs and poor response times.

Examples of resource exhaustion include the following:

- ☞ Hardware exhaustion
- ☞ Table scans in high-volume transactions causing inevitable disk I/O shortages
- ☞ Excessive network requests, resulting in network and scheduling bottlenecks
- ☞ Memory allocation causing paging and swapping
- ☞ Excessive process and thread allocation causing operating system thrashing
- This means that application designers must create a design that uses the same resources, regardless of user populations and data volumes, and does not put loads on the system resources beyond their limits.

System Scalability

- Applications that are accessible through the Internet have more complex performance and availability requirements.
- Some applications are designed and written only for Internet use, but even typical back-office applications—such as a general ledger application—might require some or all data to be available online.

Characteristics of Internet age applications include the following:

- Availability 24 hours a day, 365 days a year
- Unpredictable and imprecise number of concurrent users
- Difficulty in capacity planning
- Availability for any type of query
- Multitier architectures
- Stateless middleware
- Rapid development timescale
- Minimal time for testing
- Applications must scale with the increase of workload and also when additional hardware is added to support increasing demand. Design errors can cause the implementation to reach its maximum, regardless of additional hardware resources or re-design efforts.
- Applications are challenged by very short development timeframes with limited time for testing and evaluation. However, bad design typically means that you must later rearchitect and reimplement the system. If you deploy an application with known architectural and implementation limitations on the Internet, and if the workload exceeds the anticipated demand, then failure is a real possibility. From a business perspective, poor performance can mean a loss of customers. If Web users do not get a response in seven seconds, then the user's attention could be lost forever.
- In many cases, the cost of re-designing a system with the associated downtime costs in migrating to new implementations exceeds the costs of properly building the original system. The moral of the story is simple: design and implement with scalability in mind from the start.

Factors Preventing Scalability

When building applications, designers and architects should aim for as close to perfect scalability as possible. This is sometimes called linear scalability, where system throughput is directly proportional to the number of CPUs.

In the real world, linear scalability is impossible for reasons beyond a designer's control. However, making the application design and implementation as scalable as possible should ensure that current and future performance objectives can be achieved through expansion of hardware components and the evolution of CPU technology.

Factors that may prevent linear scalability include:

- Poor application design, implementation, and configuration
 - The application has the biggest impact on scalability. For example:
 - Poor schema design can cause expensive SQL that do not scale.
 - Poor transaction design can cause locking and serialization problems.
 - Poor connection management can cause poor response times and unreliable systems.

However, the design is not the only problem. The physical implementation of the application can be the weak link. For example:

- Systems can move to production environments with bad I/O strategies.
 - The production environment could have different execution plans from those generated in testing.
 - Memory-intensive applications that allocate a large amount of memory without much thought for freeing the memory at run time can cause excessive memory usage.
 - Inefficient memory usage and memory leaks put a high stress on the operating virtual memory subsystem. This impacts performance and availability.
- Incorrect sizing of hardware components

- Bad capacity planning of all hardware components is becoming less of a problem as relative hardware prices decrease. However, too much capacity can mask scalability problems as the workload is increased on a system.
- Limitations of software components

All software components have scalability and resource usage limitations. This applies to application servers, database servers, and operating systems. Application design should not place demands on the software beyond what it can handle.

- Limitations of hardware components

Hardware is not perfectly scalable. Most multiprocessor computers can get close to linear scaling with a finite number of CPUs, but after a certain point each additional CPU can increase performance overall, but not proportionately. There might come a time when an additional CPU offers no increase in performance, or even degrades performance. This behavior is very closely linked to the workload and the operating system setup.

System Architecture

- There are two main parts to a system's architecture:
 - *Hardware and Software Components*
 - *Configuring the Right System Architecture for Your Requirements*

Hardware and Software Components

A system architecture mainly contains hardware and software components.

- Hardware Components
- Software Components

Hardware Components

- Today's designers and architects are responsible for sizing and capacity planning of hardware at each tier in a multitier environment. It is the architect's responsibility to achieve a balanced design. This is analogous to a bridge designer who must consider all the various payload and structural requirements for the bridge. A bridge is only as strong as its weakest component. As a result, a bridge is designed in balance, such that all components reach their design limits simultaneously.
- The following are the main hardware components of a system.

CPU

There can be one or more CPUs, and they can vary in processing power from simple CPUs found in hand-held devices to high-powered server CPUs. Sizing of other hardware components is usually a multiple of the CPUs on the system.

Memory

Database and application servers require considerable amounts of memory to cache data and avoid time-consuming disk access.

I/O Subsystem

The I/O subsystem can vary between the hard disk on a client PC and high performance disk arrays. Disk arrays can perform thousands of I/Os each second and provide availability through redundancy in terms of multiple I/O paths and hot pluggable mirrored disks.

Network

All computers in a system are connected to a network, from a modem line to a high speed internal LAN. The primary concerns with network specifications are bandwidth (volume) and latency (speed).

Software Components

The same way computers have common hardware components, applications have common functional components. By dividing software development into functional components, it is possible to better comprehend the application design and architecture. Some components of the system are performed by existing software bought to accelerate application implementation, or to avoid re-development of common components.

The difference between software components and hardware components is that while hardware components only perform one task, a piece of software can perform the roles of various software components. For example, a disk drive only stores and retrieves data, but a client program can manage the user interface and perform business logic.

Most applications involve the following software components:

User Interface

This component is the most visible to application users, and includes the following functions:

- Displaying the screen to the user
- Collecting user data and transferring it to business logic
- Validating data entry
- Navigating through levels or states of the application

Business Logic

This component implements core business rules that are central to the application function. Errors made in this component can be very costly to repair. This component is implemented by a mixture of declarative and procedural approaches. An example of a declarative activity is defining unique and foreign keys. An example of procedure-based logic is implementing a discounting strategy.

Common functions of this component include:

- Moving a data model to a relational table structure
- Defining constraints in the relational table structure
- Coding procedural logic to implement business rules

Resources for Managing User Requests

- This component is implemented in all pieces of software. However, there are some requests and resources that can be influenced by the application design and some that cannot.
- In a multiuser application, most resource allocation by user requests are handled by the database server or the operating system. However, in a large application where the number of users and their usage pattern is unknown or growing rapidly, the system architect must be proactive to ensure that no single software component becomes overloaded and unstable.
- Common functions of this component include:
 - Connection management with the database
 - Executing SQL efficiently (cursors and SQL sharing)
 - Managing client state information
 - Balancing the load of user requests across hardware resources
 - Setting operational targets for hardware and software components
 - Persistent queuing for asynchronous execution of tasks

Data and Transactions

- This component is largely the responsibility of the database server and the operating system.
- Common functions of this component include:
 - Providing concurrent access to data using locks and transactional semantics
 - Providing optimized access to the data using indexes and memory cache
 - Ensuring that data changes are logged in the event of a hardware failure
 - Enforcing any rules defined for the data

Configuring the Right System Architecture for Your Requirements

- Configuring the initial system architecture is a largely iterative process. System architects must satisfy the system requirements within budget and schedule constraints. If the system requires interactive users transacting business-making decisions based on the contents of a database, then user requirements drive the architecture. If there are few interactive users on the system, then the architecture is process-driven.

- Examples of interactive user applications:
 - Accounting and bookkeeping applications
 - Order entry systems
 - Email servers
 - Web-based retail applications
 - Trading systems
- Examples of process-driven applications:
 - Utility billing systems
 - Fraud detection systems
 - Direct mail
- In many ways, process-driven applications are easier to design than multiuser applications because the user interface element is eliminated. However, because the objectives are process-oriented, system architects not accustomed to dealing with large data volumes and different success factors can become confused. Process-driven applications draw from the skills sets used in both user-based applications and data warehousing. Therefore, this book focuses on evolving system architectures for interactive users.

Note:

Generating a system architecture is not a deterministic process. It requires careful consideration of business requirements, technology choices, existing infrastructure and systems, and actual physical resources, such as budget and manpower.

- The ***following questions should stimulate thought on system architecture, though they are not a definitive guide to system architecture.*** These questions demonstrate how business requirements can influence the architecture, ease of implementation, and overall performance and availability of a system. For example:

How many users must the system support?

Most applications fall into one of the following categories:

- Very few users on a lightly-used or exclusive computer
- For this type of application, there is usually one user. The focus of the application design is to make the single user as productive as possible

by providing good response time, yet make the application require minimal administration. Users of these applications rarely interfere with each other and have minimal resource conflicts.

- A medium to large number of users in a corporation using shared applications
- For this type of application, the users are limited by the number of employees in the corporation actually transacting business through the system. Therefore, the number of users is predictable. However, delivering a reliable service is crucial to the business. The users must share a resource, so design efforts must address response time under heavy system load, escalation of resource for each session usage, and room for future growth.
- An infinite user population distributed on the Internet
- For this type of application, extra engineering effort is required to ensure that no system component exceeds its design limits. This creates a bottleneck that halts or destabilizes the system. These applications require complex load balancing, stateless application servers, and efficient database connection management. In addition, use statistics and governors to ensure that the user receives feedback if the database cannot satisfy their requests because of system overload.

What will be the user interaction method?

The choices of user interface range from a simple Web browser to a custom client program.

Where are the users located?

The distance between users influences how the application is engineered to cope with network latencies. The location also affects which times of the day are busy, when it is impossible to perform batch or system maintenance functions.

What is the network speed?

Network speed affects the amount of data and the conversational nature of the user interface with the application and database servers. A highly conversational user interface can communicate with back-end servers on every key stroke or field level validation. A less conversational interface works on a screen-sent and a screen-received model. On a slow network, it is impossible to achieve high data entry speeds with a highly conversational user interface.

How much data will the user access, and how much of that data is largely read only?

The amount of data queried online influences all aspects of the design, from table and index design to the presentation layers. Design efforts must ensure that user response time is not a function of the size of the database. If the application is largely read only, then replication and data distribution to local caches in the application servers become a viable option. This also reduces workload on the core transactional server.

What is the user response time requirement?

Consideration of the user type is important. If the user is an executive who requires accurate information to make split second decisions, then user response time cannot be compromised. Other types of users, such as users performing data entry activities, might not need such a high level of performance.

Do users expect 24 hour service?

This is mandatory for today's Internet applications where trade is conducted 24 hours a day. However, corporate systems that run in a single time zone might be able to tolerate after-hours downtime. You can use this after-hours downtime to run batch processes or to perform system administration. In this case, it might be more economic not to run a fully-available system.

Must all changes be made in real time?

It is important to determine whether transactions must be executed within the user response time, or if they can be queued for asynchronous execution.

The following are secondary questions, which can also influence the design, but really have more impact on budget and ease of implementation. For example:

How big will the database be?

This influences the sizing of the database server. On servers with a very large database, it might be necessary to have a bigger computer than dictated by the workload. This is because the administration overhead with large databases is largely a function of the database size. As tables and indexes grow, it takes proportionately more CPUs to allow table reorganizations and index builds to complete in an acceptable time limit.

What is the required throughput of business transactions?

What are the availability requirements?

Do skills exist to build and administer this application?

What compromises are forced by budget constraints?

Application Design Principles

The following are the design decisions that are involved in building applications:

- ***Simplicity In Application Design***
- ***Data Modeling***
- ***Table and Index Design***
- ***Using Views***
- ***SQL Execution Efficiency***
- ***Implementing the Application***
- ***Trends in Application Development***

Simplicity In Application Design

Applications are no different than any other designed and engineered product. Well-designed structures, computers, and tools are usually reliable, easy to use and maintain, and simple in concept. In the most general terms, if the design looks correct, then it probably is. This principle should always be kept in mind when building applications.

Consider the following design issues:

- If the table design is so complicated that nobody can fully understand it, then the table is probably poorly designed.
- If SQL statements are so long and involved that it would be impossible for any optimizer to effectively optimize it in real time, then there is probably a bad statement, underlying transaction, or table design.
- If there are indexes on a table and the same columns are repeatedly indexed, then there is probably a poor index design.
- If queries are submitted without suitable qualification for rapid response for online users, then there is probably a poor user interface or transaction design.
- If the calls to the database are abstracted away from the application logic by many layers of software, then there is probably a bad software development method.

Data Modeling

- Data modeling is important to successful relational application design. You must perform this modeling in a way that quickly represents the business practices. Heated debates may occur about the correct data model. The important thing is to apply greatest modeling efforts to those entities affected by the most frequent business transactions. In the modeling phase, there is a great temptation to spend too much time modeling the non-core data elements, which results in increased development lead times. Use of modeling tools can then rapidly generate schema definitions and can be useful when a fast prototype is required.

Table and Index Design

- Table design is largely a compromise between flexibility and performance of core transactions. To keep the database flexible and able to accommodate unforeseen workloads, the table design should be very similar to the data model, and it should be normalized to at least 3rd normal form. However, certain core transactions required by users can require selective denormalization for performance purposes.
- Examples of this technique include storing tables pre-joined, the addition of derived columns, and aggregate values. Oracle Database provides numerous options for storage of aggregates and pre-joined data by clustering and materialized view functions. These features allow a simpler table design to be adopted initially.

- Again, focus and resources should be spent on the business critical tables, so that optimal performance can be achieved. For non-critical tables, shortcuts in design can be adopted to enable a more rapid application development. However, if prototyping and testing a non-core table becomes a performance problem, then remedial design effort should be applied immediately.
- Index design is also a largely iterative process, based on the SQL generated by application designers. However, it is possible to make a sensible start by building indexes that enforce primary key constraints and indexes on known access patterns, such as a person's name. As the application evolves, and as you perform testing on realistic amounts of data, you may need to improve the performance of specific queries by building a better index. Consider the following list of indexing design ideas when building a new index:
 - [Appending Columns to an Index or Using Index-Organized Tables](#)
 - [Using a Different Index Type](#)
 - [Finding the Cost of an Index](#)
 - [Serializing within Indexes](#)
 - [Ordering Columns in an Index](#)

Using Views

- Views can speed up and simplify application design. A simple view definition can mask data model complexity from the programmers whose priorities are to retrieve, display, collect, and store data.
- However, while views provide clean programming interfaces, they can cause sub-optimal, resource-intensive queries. The worst type of view use is when a view references other views, and when they are joined in queries. In many cases, developers can satisfy the query directly from the table without using a view. Usually, because of their inherent properties, views make it difficult for the optimizer to generate the optimal execution plan.

SQL Execution Efficiency

In the design and architecture phase of any system development, care should be taken to ensure that the application developers understand SQL execution efficiency. To achieve this goal, the development environment must support the following characteristics:

- Good database connection management

Connecting to the database is an expensive operation that is highly unscalable. Therefore, the number of concurrent connections to the database should be minimized as much as possible. A simple system, where a user connects at application initialization, is ideal. However, in a Web-based or multitiered application, where application servers are used to multiplex database connections to users, this can be difficult. With these types of applications, design efforts should ensure that database connections are pooled and are not reestablished for each user request.

- Good cursor usage and management

Maintaining user connections is equally important to minimizing the parsing activity on the system. Parsing is the process of interpreting a SQL statement and creating an execution plan for it. This process has many phases, including syntax checking, security checking, execution plan generation, and loading shared structures into the shared pool. There are two types of parse operations:

- Hard parsing

A SQL statement is submitted for the first time, and no match is found in the shared pool. Hard parses are the most resource-intensive and unscalable, because they perform all the operations involved in a parse.

- Soft parsing

A SQL statement is submitted for the first time, and a match is found in the shared pool. The match can be the result of previous execution by another user. The SQL statement is shared, which is good for performance. However, soft parses are not ideal, because they still require syntax and security checking, which consume system resources.

- Because parsing should be minimized as much as possible, application developers should design their applications to parse SQL statements once and execute them many

times. This is done through cursors. Experienced SQL programmers should be familiar with the concept of opening and re-executing cursors.

Application developers must also ensure that SQL statements are shared within the shared pool. To achieve this goal, use bind variables to represent the parts of the query that change from execution to execution. If this is not done, then the SQL statement is likely to be parsed once and never re-used by other users. To ensure that SQL is shared, use bind variables and do not use string literals with SQL statements. For example:

Statement with string literals:

```
SELECT * FROM employees

WHERE last_name LIKE 'KING';
```

Statement with bind variables:

```
SELECT * FROM employees

WHERE last_name LIKE :1;
```

The following example shows the results of some tests on a simple OLTP application:

Test	#Users Supported
No Parsing all statements	270
Soft Parsing all statements	150
Hard Parsing all statements	60
Re-Connecting for each Transaction	30

These tests were performed on a four-CPU computer. The differences increase as the number of CPUs on the system increase.

Implementing the Application

The choice of development environment and programming language is largely a function of the skills available in the development team and architectural decisions made when specifying the application. There are, however, some simple performance management rules that can lead to scalable, high-performance applications.

1. Choose a development environment suitable for software components, and do not let it limit your design for performance decisions. If it does, then you probably chose the wrong language or environment.

- **User interface**

The programming model can vary between HTML generation and calling the windowing system directly. The development method should focus on response time of the user interface code. If HTML or Java is being sent over a network, then try to minimize network volume and interactions.

- **Business logic**

Interpreted languages, such as Java and PL/SQL, are ideal to encode business logic. They are fully portable, which makes upgrading logic relatively easy. Both languages are syntactically rich to allow code that is easy to read and interpret. If business logic requires complex mathematical functions, then a compiled binary language might be needed. The business logic code can be on the client computer, the application server, and the database server. However, the application server is the most common location for business logic.

- **User requests and resource allocation**

Most of this is not affected by the programming language, but tools and fourth generation languages that mask database connection and cursor management might use inefficient mechanisms. When evaluating these tools and environments, check their database connection model and their use of cursors and bind variables.

- **Data management and transactions**

Most of this is not affected by the programming language.

2. When implementing a software component, implement its function and not the functionality associated with other components. Implementing another component's functionality results in sub-optimal designs and implementations. This applies to all components.
3. Do not leave gaps in functionality or have software components under-researched in design, implementation, or testing. In many cases, gaps are not discovered until the application is rolled out or tested at realistic volumes. This is usually a sign of poor architecture or initial system specification. Data archival and purge modules are most frequently neglected during initial system design, build, and implementation.
4. When implementing procedural logic, implement in a procedural language, such as C, Java, or PL/SQL. When implementing data access (queries) or data changes (DML), use SQL. This rule is specific to the business logic modules of code where procedural code is mixed with data access (nonprocedural SQL) code. There is great temptation to put procedural logic into the SQL access. This tends to result in poor SQL that is resource-intensive. SQL statements with DECODE case statements are very often candidates for optimization, as are statements with a large amount of OR predicates or set operators, such as UNION and MINUS.
5. Cache frequently accessed, rarely changing data that is expensive to retrieve on a repeated basis. However, make this cache mechanism easy to use, and ensure that it is indeed cheaper than accessing the data in the original method. This is applicable to all modules where frequently used data values should be cached or stored locally, rather than be repeatedly retrieved from a remote or expensive data store.

The most common examples of candidates for local caching include the following:

- Today's date. `SELECT SYSDATE FROM DUAL` can account for over 60% of the workload on a database.
- The current user name.
- Repeated application variables and constants, such as tax rates, discounting rates, or location information.

- Caching data locally can be further extended into building a local data cache into the application server middle tiers. This helps take load off the central database servers. However, care should be taken when constructing local caches so that they do not become so complex that they cease to give a performance gain.
- Local sequence generation.

The design implications of using a cache should be considered. For example, if a user is connected at midnight and the date is cached, then the user's date value becomes invalid.

6. Optimize the interfaces between components, and ensure that all components are used in the most scalable configuration. This rule requires minimal explanation and applies to all modules and their interfaces.
7. Use foreign key references. Enforcing referential integrity through an application is expensive. You can maintain a foreign key reference by selecting the column value of the child from the parent and ensuring that it exists. The foreign key constraint enforcement supplied by Oracle—which does not use SQL—is fast, easy to declare, and does not create network traffic.
8. Consider setting up action and module names in the application to use with End to End Application Tracing. This allows greater flexibility in tracing workload problems.

Trends in Application Development

- The two biggest challenges in application development today are the increased use of Java to replace compiled C or C++ applications, and increased use of object-oriented techniques, influencing the schema design.
- Java provides better portability of code and availability to programmers. However, there are several performance implications associated with Java. Because Java is an interpreted language, it is slower at executing similar logic than compiled languages, such as C. As a result, resource usage of client computers increases. This requires more powerful CPUs to be applied in the client or middle-tier computers and greater care from programmers to produce efficient code.
- Because Java is an object-oriented language, it encourages insulation of data access into classes not performing the business logic. As a result, programmers might

invoke methods without knowledge of the efficiency of the data access method being used. This tends to result in minimal database access and uses the simplest and crudest interfaces to the database.

- With this type of software design, queries do not always include all the WHERE predicates to be efficient, and row filtering is performed in the Java program. This is very inefficient. In addition, for DML operations—and especially for INSERTs—single INSERTs are performed, making use of the array interface impossible. In some cases, this is made more inefficient by procedure calls. More resources are used moving the data to and from the database than in the actual database calls.
- In general, it is best to place data access calls next to the business logic to achieve the best overall transaction design.
- The acceptance of object-orientation at a programming level has led to the creation of object-oriented databases within the Oracle Server. This has manifested itself in many ways, from storing object structures within BLOBs and only using the database effectively as an indexed card file to the use of the Oracle Database object-relational features.
- If you adopt an object-oriented approach to schema design, then ensure that you do not lose the flexibility of the relational storage model. In many cases, the object-oriented approach to schema design ends up in a heavily denormalized data structure that requires considerable maintenance and REF pointers associated with objects. Often, these designs represent a step backward to the hierarchical and network database designs that were replaced with the relational storage method.
- In summary, if you are storing your data in your database for the long-term, and if you anticipate a degree of ad hoc queries or application development on the same schema, then the relational storage method probably gives the best performance and flexibility.

Workload Testing, Modeling, and Implementation

This includes the following:

- Sizing Data
- Estimating Workloads
- Application Modeling

- Testing, Debugging, and Validating a Design

Sizing Data

You could experience errors in your sizing estimates when dealing with variable length data if you work with a poor sample set. As data volumes grow, your key lengths could grow considerably, altering your assumptions for column sizes.

When the system becomes operational, it becomes more difficult to predict database growth, especially for indexes. Tables grow over time, and indexes are subject to the individual behavior of the application in terms of key generation, insertion pattern, and deletion of rows. The worst case is where you insert using an ascending key, and then delete most rows from the left-hand side but not all the rows. This leaves gaps and wasted space. If you have index use like this, then ensure that you know how to use the online index rebuild facility.

DBAs should monitor space allocation for each object and look for objects that may grow out of control. A good understanding of the application can highlight objects that may grow rapidly or unpredictably. This is a crucial part of both performance and availability planning for any system. When implementing the production database, the design should attempt to ensure that minimal space management takes place when interactive users are using the application. This applies for all data, temp, and rollback segments.

Estimating Workloads

Considering the number of variables involved, estimation of workloads for capacity planning and testing purposes is extremely difficult. However, designers must specify computers with CPUs, memory, and disk drives, and eventually roll out an application. There are several techniques used for sizing, and each technique has merit. When sizing, it is best to use the following methods to validate your decision-making process and provide supporting documentation.

Extrapolating From a Similar System

This is an entirely empirical approach where an existing system of similar characteristics and known performance is used as a basis system. The specification of this system is then modified by the sizing specialist according to the known differences. This approach has

merit in that it correlates with an existing system, but it provides little assistance when dealing with the differences.

This approach is used in nearly all large engineering disciplines when preparing the cost of an engineering project, such as a large building, a ship, a bridge, or an oil rig. If the reference system is an order of magnitude different in size from the anticipated system, then some components may have exceeded their design limits.

Benchmarking

The benchmarking process is both resource and time consuming, and it might not produce the correct results. By simulating an application in early development or prototype form, there is a danger of measuring something that has no resemblance to the actual production system. This sounds strange, but over the many years of benchmarking customer applications with the database development organization, Oracle has yet to see reliable correlation between the benchmark application and the actual production system. This is mainly due to the number of application inefficiencies introduced in the development process.

However, benchmarks have been used successfully to size systems to an acceptable level of accuracy. In particular, benchmarks are very good at determining the actual I/O requirements and testing recovery processes when a system is fully loaded.

Benchmarks by their nature stress all system components to their limits. As the benchmark stresses all components, be prepared to see all errors in application design and implementation manifest themselves while benchmarking. Benchmarks also test database, operating system, and hardware components. Because most benchmarks are performed in a rush, expect setbacks and problems when a system component fails. Benchmarking is a stressful activity, and it takes considerable experience to get the most out of a benchmarking exercise.

Application Modeling

Modeling the application can range from complex mathematical modeling exercises to the classic simple calculations performed on the back of an envelope. Both methods have merit,

with one attempting to be very precise and the other making gross estimates. The downside of both methods is that they do not allow for implementation errors and inefficiencies.

The estimation and sizing process is an imprecise science. However, by investigating the process, some intelligent estimates can be made. The whole estimation process makes no allowances for application inefficiencies introduced by poor SQL, index design, or cursor management. A sizing engineer should build in margin for application inefficiencies. A performance engineer should discover the inefficiencies and make the estimates look realistic. The Oracle performance method describes how to discover the application inefficiencies.

Testing, Debugging, and Validating a Design

The testing process mainly consists of functional and stability testing. At some point in the process, performance testing is performed.

The following list describes some simple rules for performance testing an application. If correctly documented, then this list provides important information for the production application and the capacity planning process after the application has gone live.

- Use the Automatic Database Diagnostic Monitor (ADDM) and SQL Tuning Advisor for design validation
- Test with realistic data volumes and distributions
- All testing must be done with fully populated tables. The test database should contain data representative of the production system in terms of data volume and cardinality between tables. All the production indexes should be built and the schema statistics should be populated correctly.
- Use the correct optimizer mode
- Perform all testing with the optimizer mode that you plan to use in production. All Oracle Database research and development effort is focused on the query optimizer. Therefore, the use of the query optimizer is recommended.
- Test a single user performance
- Test a single user on an idle or lightly-used database for acceptable performance. If a single user cannot achieve acceptable performance under ideal conditions, then multiple users cannot achieve acceptable performance under real conditions.

- Obtain and document plans for all SQL statements

Obtain an execution plan for each SQL statement. Use this process to verify that the optimizer is obtaining an optimal execution plan, and that the relative cost of the SQL statement is understood in terms of CPU time and physical I/Os. This process assists in identifying the heavy use transactions that require the most tuning and performance work in the future.

- Attempt multiuser testing

This process is difficult to perform accurately, because user workload and profiles might not be fully quantified. However, transactions performing DML statements should be tested to ensure that there are no locking conflicts or serialization problems.

- Test with the correct hardware configuration

Test with a configuration as close to the production system as possible. Using a realistic system is particularly important for network latencies, I/O subsystem bandwidth, and processor type and speed. Failing to use this approach may result in an incorrect analysis of potential performance problems.

- Measure steady state performance

When benchmarking, it is important to measure the performance under steady state conditions. Each benchmark run should have a ramp-up phase, where users are connected to the application and gradually start performing work on the application. This process allows for frequently cached data to be initialized into the cache and single execution operations—such as parsing—to be completed before the steady state condition. Likewise, at the end of a benchmark run, there should be a ramp-down period, where resources are freed from the system and users cease work and disconnect.

Deploying New Applications

The following are the key design decisions involved in deploying applications:

- Rollout Strategies
- Performance Checklist

Rollout Strategies

- When new applications are rolled out, two strategies are commonly adopted:
 - *Big Bang approach* - all users migrate to the new system at once
 - *Trickle approach* - users slowly migrate from existing systems to the new one
- Both approaches have merits and disadvantages. The Big Bang approach relies on reliable testing of the application at the required scale, but has the advantage of minimal data conversion and synchronization with the old system, because it is simply switched off. The Trickle approach allows debugging of scalability issues as the workload increases, but might mean that data must be migrated to and from legacy systems as the transition takes place.
- It is difficult to recommend one approach over the other, because each method has associated risks that could lead to system outages as the transition takes place. Certainly, the Trickle approach allows profiling of real users as they are introduced to the new application, and allows the system to be reconfigured while only affecting the migrated users. This approach affects the work of the early adopters, but limits the load on support services. This means that unscheduled outages only affect a small percentage of the user population.
- The decision on how to roll out a new application is specific to each business. Any adopted approach has its own unique pressures and stresses. The more testing and knowledge that you derive from the testing process, the more you realize what is best for the rollout.

Performance Checklist

To assist in the rollout, build a list of tasks that increase the chance of optimal performance in production and enable rapid debugging of the application. Do the following:

1. When you create the control file for the production database, allow for growth by setting MAXINSTANCES, MAXDATAFILES, MAXLOGFILES, MAXLOGMEMBERS, and MAXLOGHISTORY to values higher than what you anticipate for the rollout. This technique results in more disk space usage and larger control files, but saves time later should these need extension in an emergency.
2. Set block size to the value used to develop the application. Export the schema statistics from the development or test environment to the production database if

the testing was done on representative data volumes and the current SQL execution plans are correct.

3. Set the minimal number of initialization parameters. Ideally, most other parameters should be left at default. If there is more tuning to perform, then this appears when the system is under load.
4. Be prepared to manage block contention by setting storage options of database objects. Tables and indexes that experience high INSERT/UPDATE/DELETE rates should be created with automatic segment space management. To avoid contention of rollback segments, use automatic undo management.
5. All SQL statements should be verified to be optimal and their resource usage understood.
6. Validate that middleware and programs that connect to the database are efficient in their connection management and do not logon or logoff repeatedly.
7. Validate that the SQL statements use cursors efficiently. The database should parse each SQL statement once and then execute it multiple times. The most common reason this does not happen is because bind variables are not used properly and WHERE clause predicates are sent as string literals. If you use precompilers to develop the application, then make sure to reset the parameters MAXOPENCURSORS, HOLD_CURSOR, and RELEASE_CURSOR from the default values before precompiling the application.
8. Validate that all schema objects have been correctly migrated from the development environment to the production database. This includes tables, indexes, sequences, triggers, packages, procedures, functions, Java objects, synonyms, grants, and views. Ensure that any modifications made in testing are made to the production system.
9. As soon as the system is rolled out, establish a baseline set of statistics from the database and operating system. This first set of statistics validates or corrects any assumptions made in the design and rollout process.
10. Start anticipating the first bottleneck (which is inevitable) and follow the Oracle performance method to make performance improvement.

Instance Tuning

- In Oracle, "instance tuning" refers to optimizing the configuration and performance of the Oracle instance, which includes memory allocation, CPU usage, disk I/O, and other resources. Instance tuning is essential for ensuring that the Oracle database performs efficiently and meets the required service levels.
- When considering instance tuning, take care in the initial design of the database to avoid bottlenecks that could lead to performance problems. In addition, you must consider:
 - Allocating memory to database structures
 - Determining I/O requirements of different parts of the database
 - Tuning the operating system for optimal performance of the database
- After the database instance has been installed and configured, you must monitor the database as it is running to check for performance-related problems.

Performance Principles

- Performance tuning requires a different, although related, method to the initial configuration of a system. Configuring a system involves allocating resources in an ordered manner so that the initial system configuration is functional.
- Tuning is driven by identifying the most significant bottleneck and making the appropriate changes to reduce or eliminate the effect of that bottleneck. Usually, tuning is performed reactively, either while the system is in preproduction or after it is live.

Baselines

- The most effective way to tune is to have an established performance baseline that you can use for comparison if a performance issue arises. Most database administrators (DBAs) know their system well and can easily identify peak usage periods. For example, the peak periods could be between 10.00am and 12.00pm and also between 1.30pm and 3.00pm. This could include a batch window of 12.00am midnight to 6am.
- It is important to identify these peak periods at the site and install a monitoring tool that gathers performance data for those high-load times. Optimally, data gathering should be configured from when the application is in its initial trial phase during

the QA cycle. Otherwise, this should be configured when the system is first in production.

- Ideally, baseline data gathered should include the following:
 - Application statistics (transaction volumes, response time)
 - Database statistics
 - Operating system statistics
 - Disk I/O statistics
 - Network statistics
- In the Automatic Workload Repository, baselines are identified by a range of snapshots that are preserved for future comparisons.

The Symptoms and the Problems

A common pitfall in performance tuning is to mistake the symptoms of a problem for the actual problem itself. It is important to recognize that many performance statistics indicate the symptoms, and that identifying the symptom is not sufficient data to implement a remedy. For example:

Slow physical I/O

Generally, this is caused by poorly-configured disks. However, it could also be caused by a significant amount of unnecessary physical I/O on those disks issued by poorly-tuned SQL.

Latch contention

Rarely is latch contention tunable by reconfiguring the instance. Rather, latch contention usually is resolved through application changes.

Excessive CPU usage

Excessive CPU usage usually means that there is little idle CPU on the system. This could be caused by an inadequately-sized system, by untuned SQL statements, or by inefficient application programs.

When to Tune

There are two distinct types of tuning:

- *Proactive Monitoring*
- *Bottleneck Elimination*

Proactive Monitoring

- Proactive monitoring usually occurs on a regularly scheduled interval, where several performance statistics are examined to identify whether the system behavior and resource usage has changed. Proactive monitoring can also be considered as proactive tuning.
- Usually, monitoring does not result in configuration changes to the system, unless the monitoring exposes a serious problem that is developing. In some situations, experienced performance engineers can identify potential problems through statistics alone, although accompanying performance degradation is usual.
- Experimenting with or tweaking a system when there is no apparent performance degradation as a proactive action can be a dangerous activity, resulting in unnecessary performance drops. Tweaking a system should be considered reactive tuning, and the steps for reactive tuning should be followed.
- Monitoring is usually part of a larger capacity planning exercise, where resource consumption is examined to see changes in the way the application is being used, and the way the application is using the database and host resources.

Bottleneck Elimination

- Tuning usually implies fixing a performance problem. However, tuning should be part of the life cycle of an application—through the analysis, design, coding, production, and maintenance stages. Often, the tuning phase is left until the database is in production. At this time, tuning becomes a reactive process, where the most important bottleneck is identified and fixed.
- Usually, the purpose for tuning is to reduce resource consumption or to reduce the elapsed time for an operation to complete. Either way, the goal is to improve the effective use of a particular resource. In general, performance problems are caused

by the overuse of a particular resource. The overused resource is the bottleneck in the system. There are several distinct phases in identifying the bottleneck and the potential fixes. These are discussed in the sections that follow.

Remember that the different forms of contention are symptoms that can be fixed by making changes in the following places:

- Changes in the application, or the way the application is used
- Changes in Oracle
- Changes in the host hardware configuration

Often, the most effective way of resolving a bottleneck is to change the application.

Performance Tuning Methodology

- Methodology is key to success in performance tuning. Different tuning strategies offer diminishing returns, and it is important to use the strategies with the maximum gains first. Furthermore, systems with different purposes, such as online transaction processing systems and decision support systems, may require different approaches.

When Is Tuning Most Effective?

- For dramatically better results, tune during the design phase rather than waiting to tune after implementing your system.
 - *Proactive Tuning While Designing and Developing a System*
 - *Reactive Tuning to Improve a Production System*
- By far the most effective approach to tuning is to work proactively.
- Business executives must collaborate with application designers to establish justifiable performance goals and set realistic performance expectations from the start. During design and development, the application designers can then determine which combination of system resources and available Oracle features will best meet these needs.
- By designing a system to perform well, you can minimize its eventual cost and frustration.

Reactive Tuning to Improve a Production System

- Many people believe the tuning process begins when users complain about poor response time. This is usually too late in the process to use some of the most effective tuning strategies. At that point, if you are unwilling to completely redesign the application, you may only improve performance marginally by reallocating memory and tuning I/O.
- Consider, for example, a bank which employs one teller and one manager. It has a business rule that the manager must approve any withdrawals exceeding \$20. Upon investigation, you may find that there is a long queue of customers, and deduce that you need more tellers. You may add 10 more tellers, but then find that the bottleneck moves to the manager's function. However, the bank may determine that it is too expensive to hire additional managers. Regardless of how carefully you may tune the system using the existing business rule, getting better performance will be very expensive.
- Upon stepping back, you may see that a change to the business rule may be necessary to make the system more scalable. If you change the rule such that the manager need only approve withdrawals exceeding \$150, you have come up with a scalable solution. In this situation, effective tuning could only be done at the highest design level, rather than at the end of the process.
- It is nonetheless possible to work reactively to tune an existing production system. To take this approach, start at the bottom of the method and work your way up, finding and fixing any bottlenecks. A common goal is to make Oracle run faster on the given platform. You may find, however, that both Oracle Server and the operating system are working well: to get additional performance gains you may have to tune the application or add resources. Only then can you take full advantage of the many features Oracle provides that can greatly improve performance when properly used in a well-designed system.
- Note that even the performance of well-designed systems can degrade with use. Ongoing tuning is therefore an important part of proper system maintenance.

Prioritized Steps of the Tuning Method

The recommended method for tuning an Oracle database prioritizes steps in order of diminishing returns: steps with the greatest impact on performance are listed first. For optimal results, therefore, tackle tuning issues in the order listed: from the design and development phases through instance tuning.

Step 1: Tune the Business Rules

Step 2: Tune the Data Design

Step 3: Tune the Application Design

Step 4: Tune the Logical Structure of the Database

Step 5: Tune Database Operations

Step 6: Tune the Access Paths

Step 7: Tune Memory Allocation

Step 8: Tune I/O and Physical Structure

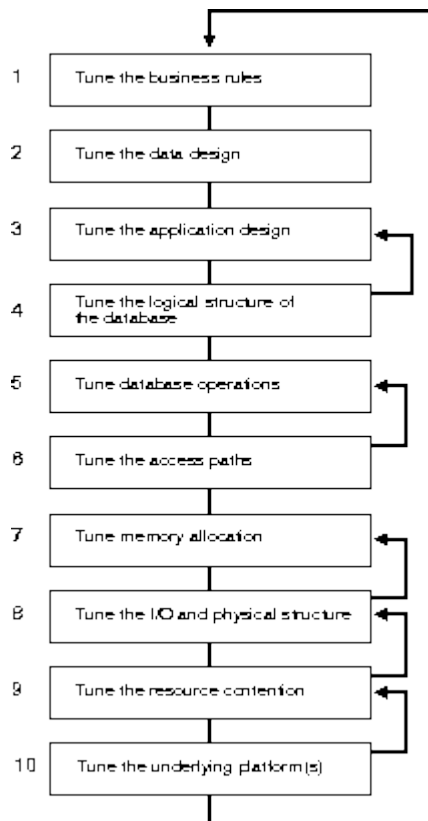
Step 9: Tune Resource Contention

Step 10: Tune the Underlying Platform(s)

After completing the steps of the tuning method, reassess your database performance and decide whether further tuning is necessary.

Note that this is an iterative process. Performance gains made in later steps may pave the way for further improvements in earlier steps, so additional passes through the tuning process may be useful.

Following figure illustrates the tuning method:



Decisions you make in one step may influence subsequent steps. For example, in step 5 you may rewrite some of your SQL statements. These SQL statements may have significant bearing on parsing and caching issues addressed in step 7. Also, disk I/O, which is tuned in step 8, depends on the size of the buffer cache, which is tuned in step 7. Although the figure shows a loop back to step 1, you may need to loop back from any step to any previous step.

Step 1: Tune the Business Rules

For optimal performance, you may have to adapt business rules. These concern the high-level analysis and design of an entire system. Configuration issues are considered at this level, such as whether or not to use a multithreaded server system-wide. In this way, the planners ensure that the performance requirements of the system correspond directly to concrete business needs.

Performance problems encountered by the DBA may actually be caused by problems in design and implementation, or by inappropriate business rules. People commonly get in too deep when they write the business functions of an application. They document an implementation, rather than simply the function that must be performed. If business executives use care in abstracting the business function or requirement from the

implementation, then designers have a wider field from which to choose the appropriate implementation.

Consider, for example, the business function of check printing. The actual requirement is to pay money to people; the requirement is not necessarily to print up pieces of paper. Whereas it would be very difficult to print up a million checks per day, it would be relatively easy to record that many direct deposit payments on a tape which could be sent to the bank for processing.

Business rules should be consistent with realistic expectations for the number of concurrent users, the transaction response time, and the number of records stored online that the system can support. For example, it would not make sense to run a highly interactive application over slow wide area network lines.

Similarly, a company soliciting users for an Internet service might advertise 10 free hours per month for all new subscribers. If 50,000 users per day signed up for this service, the demand would far exceed the capacity for a client/server configuration. The company should instead consider using a multitier configuration. In addition, the signup process must be simple: it should require only one connection from the user to the database, or connection to multiple databases without dedicated connections, making use of a multithreaded server or transaction monitor approach.

Step 2: Tune the Data Design

In the data design phase, you must determine what data is needed by your applications. You need to consider what relations are important, and what their attributes are. Finally you need to structure the information to best meet performance goals.

The database design process generally undergoes a normalization stage, in which data is analyzed to ensure that no redundant data will be held anywhere. One fact should be stated in one and only one place in the database. Once the data is carefully normalized, however, you may need to denormalize it for performance reasons. You might, for example, decide that the database should hold frequently required summary values. Rather than forcing an application to recalculate the total price of all the lines in a given order each time it is

accessed, you might decide to include the total value of each order in the database. You could set up primary key and foreign key indexes to access this information quickly.

Another data design consideration is the avoidance of contention on data. Consider a database 1 terabyte in size, on which a thousand users access only 0.5% of the data. This "hot spot" in the data could cause performance problems.

Try also to localize access to the data: localize it to each process, to each instance, and to each partition. Contention begins when access becomes remote, and the amount of contention determines scalability.

In Oracle Parallel Server, look for synchronization points-any point in time, or part of an application, that must run sequentially, one process at a time. The requirement of having sequential order numbers, for example, is a synchronization point that results from poor design.

Consider two Oracle8 enhancements that can help you to tune the data design to avoid contention:

- Consider whether or not to partition your data.
- Consider whether to use local or global indexes.

Step 3: Tune the Application Design

Business executives and application designers need to translate business goals into an effective system design. Business processes concern a particular application within a system, or a particular part of an application.

An example of intelligent process design is strategically caching data. For example, in a retail application you can select the tax rate once at the beginning of each day, and cache it within the application. In this way you avoid retrieving the same information over and over during the course of the day.

At this level also, you can consider configuration of individual processes. For example, some PC users may be accessing the central system using mobile agents, whereas other users may be directly connected. Although they are running on the same system, the

architecture is different. They may also require different mail servers and different versions of the application.

Step 4: Tune the Logical Structure of the Database

After the application and the system have been designed, you can plan the logical structure of the database. This primarily concerns fine-tuning the index design, to ensure that the data is neither over- nor under-indexed. In the data design stage (Step 2) you determine the primary and foreign key indexes. In the logical structure design stage you may create additional indexes to support the application.

Performance problems due to contention often involve inserts into the same block or incorrect use of sequence numbers. Use particular care in designing the use and location of indexes, the sequence generator, and clusters.

Step 5: Tune Database Operations

System designers and application developers must understand Oracle's query processing mechanism to write effective SQL statements.

Before tuning the Oracle Server itself, be certain that your application is taking full advantage of the SQL language and the Oracle features designed to speed application processing. Use features and techniques such as the following based on the needs of your application:

- Array processing
- The Oracle optimizer
- The row-level lock manager
- PL/SQL

Step 6: Tune the Access Paths

Ensure that there is efficient access to data. Consider the use of clusters, hash clusters, B*-tree indexes and bitmap indexes.

Ensuring efficient access may mean adding indexes, or adding indexes for a particular application and then dropping them again. It may mean revisiting your design after you have built the database. You may want to do more normalization or create alternative indexes at this point. Upon testing the application you may find that you're still not obtaining the required response time. Look for more ways to improve the design.

Step 7: Tune Memory Allocation

Appropriate allocation of memory resources to Oracle memory structures can have a large impact on performance.

Oracle shared memory is allocated dynamically to the following structures, which are all part of the shared pool. Although you explicitly set the total amount of memory available in the shared pool, the system dynamically sets the size of each structure contained within it:

- The data dictionary cache
- The library cache
- Context areas (if running a multithreaded server)

You can explicitly set memory allocation for the following structures:

- Buffer cache
- Log buffer
- Sequence caches

Proper allocation of memory resources can improve cache performance, reduce parsing of SQL statements, and reduce paging and swapping.

Process local areas include:

- Context areas (for systems not running a multithreaded server)
- Sort areas
- Hash areas

Be careful not to allocate to the system global area (SGA) such a large percentage of the machine's physical memory that it causes paging or swapping.

Step 8: Tune I/O and Physical Structure

Disk I/O tends to reduce the performance of many software applications. Oracle Server, however, is designed so that its performance need not be unduly limited by I/O. Tuning I/O and physical structure involves these procedures:

- Distributing data so that I/O is distributed, thus avoiding disk contention
- Storing data in data blocks for best access: setting the right number of free lists, and proper values for PCTFREE and PCTUSED
- Creating extents large enough for your data so as to avoid dynamic extension of tables, which would hurt high-volume OLTP applications
- Evaluating the use of raw devices

Step 9: Tune Resource Contention

Concurrent processing by multiple Oracle users may create contention for Oracle resources. Contention may cause processes to wait until resources are available. Take care to reduce the following kinds of contention:

- Block contention
- Shared pool contention
- Lock contention
- Pinging (in a parallel server environment)
- Latch contention

Step 10: Tune the Underlying Platform(s)

See your platform-specific Oracle documentation to investigate ways of tuning the underlying system. For example, on UNIX-based systems you might want to tune the following:

- Size of the UNIX buffer cache
- Logical volume managers

- Memory and size for each process

How to Apply the Tuning Method

Follow these for applying Tuning Methods

- *Set Clear Goals for Tuning*
- *Create Minimum Repeatable Tests*
- *Test Hypotheses*
- *Keep Records*
- *Avoid Common Errors*
- *Stop Tuning When the Objectives Are Met*
- *Demonstrate Meeting the Objectives*

Set Clear Goals for Tuning

Never begin tuning without having first established clear objectives: you cannot succeed if there is no definition of "success."

"Just make it go as fast as you can" may sound like an objective, but it will be very difficult to determine whether this has been achieved. It will be even more difficult to tell whether your results have met the underlying business requirements. A more useful statement of objectives is the following: "We need to have as many as 20 operators each entering 20 orders per hour, and the packing lists produced within 30 minutes of the end of the shift."

Keep your goals in mind as you consider each tuning measure; consider its performance benefits in light of your goals.

Also bear in mind that your goals may conflict. For example, to achieve best performance for a specific SQL statement, you may have to sacrifice the performance of other SQL statements running concurrently on your database.

Create Minimum Repeatable Tests

Create a series of minimum reproducible cases. For example, if you manage to identify a single SQL statement that is causing a performance problem, then run both the original and

the revised version of that statement in SQL*Plus (with the SQL trace facility or Oracle Trace enabled) so that you can see statistically the difference in performance. In many cases, a tuning effort can succeed simply by identifying one SQL statement that was causing the performance problem.

If you must cut a 4-hour run down to 2 hours duration, you will probably find that repeated timings take too long. Perform your initial trials against a test environment that exhibits a profile similar to the real one. For example, you could impose some additional restrictive condition such as processing one department instead of all 500 of them. The ideal test case will run for more than 1 minute, so that improvements can be seen intuitively, as well as measured using timing features. It should run for less than 5 minutes, however, so that test execution does not consume an excessive proportion of the time available.

Test Hypotheses

With a minimum repeatable test established, and with a script both to conduct the test and to summarize and report the results, you can test various hypotheses to see the effect.

Bear in mind that Oracle's caching algorithms mean that the first time data is visited there is an additional overhead. Thus, if two approaches are tried one after the other, the second will always have a tactical advantage: data which it would otherwise have had to read from disk may be left in the cache.

Keep Records

Keep records of the effect of each change: incorporate record keeping in the script being used to run the test. You should automate the testing process for a number of reasons:

- For cost effectiveness in terms of the tuner's ability to conduct tests quickly
- To ensure that the tests are conducted in the same way, using the same instrumentation, for each hypothesis being tested

Anecdotal results from tests should be checked against the objective data before being accepted.

Avoid Common Errors

A common error made by inexperienced tuners is to cling to preconceived notions about what may be causing the problem at hand. The next most common error is to try various approaches at random.

Each time you think that you are onto something, try explaining it to someone else. Often you yourself will spot mistakes, simply from having gone through the discipline of articulating your ideas. For best results you should build a team of people to resolve performance problems. While a performance tuner can tune SQL statements without knowing the application in detail, the team should include someone who does understand the application and who can validate the solutions that the SQL tuner may devise.

Avoid Rash Actions

Beware of doing something rash. Once you have a hypothesis, you may be tempted to implement it globally throughout the system and then wait to see the results. You can ruin a perfectly good system in this way!

Avoid Preconceptions

Try to avoid preconceptions when you come to a tuning problem. Get users to tell you the symptoms they perceive-but do not expect them to know why the problem exists.

One user, for example, had serious system memory problems over a long period of time. In the morning the system ran well, but performance then dropped off very rapidly. A consultant called in to tune the system was told that a PL/SQL memory leak was the cause. As it turned out, this was not at all the problem. Rather, the user had set `SORT_AREA_SIZE` to 10 MB on a machine with 64 MB of memory, and had 20 users. When users came on to the system, the first time they did a sort they would get their sort area. The system thus was burdened with 200 MB of virtual memory and was hopelessly swapping and paging.

Many people will speculate about the cause of the problem. Ask questions of those affected, and of those responsible for the system. Listen to the symptoms that users describe, but do not accept prima facie their notions as to the cause!

Avoid "Hit and Hope"

Avoid seizing on panaceas that may be bandied about in database folklore, such as "set the GO_FASTER parameter and everything will work faster." Be wary of apocryphal tales—such as the false notion that all tables must be in a single extent for performance to be acceptable.

Stop Tuning When the Objectives Are Met

One of the great advantages of having targets for tuning is that it becomes possible to define success. Past a certain point, it is no longer cost effective to continue tuning a system.

Demonstrate Meeting the Objectives

As the tuner you may be confident that the performance targets have been met. You nonetheless must demonstrate this to two communities:

- the users affected by the problem
- those responsible for the application's success

Performance Tuning Data

- Performance tuning in Oracle involves analyzing various data points to identify bottlenecks and areas for optimization. Here are some key types of data commonly used in Oracle performance tuning:
 - **Wait Events:** Oracle tracks wait events, which represent the time spent by sessions waiting for various resources, such as I/O, locks, latch acquisition, and network communication. Monitoring wait events helps identify performance bottlenecks and areas for optimization.
 - **System and Session Statistics:** Oracle provides a wide range of system and session-level statistics that can be monitored to gauge the overall health and

performance of the database. This includes metrics such as CPU usage, memory utilization, I/O rates, and database buffer cache hit ratios.

- **Execution Plans:** SQL execution plans show the steps and operations performed by Oracle's optimizer to execute a SQL statement. Analyzing execution plans helps identify inefficient queries, suboptimal index usage, and opportunities for optimization.
- **Buffer Cache Statistics:** Monitoring buffer cache statistics, such as cache hit ratios and buffer busy waits, provides insights into how effectively the database is utilizing memory for caching data blocks. Optimizing buffer cache usage can improve overall database performance.
- **SQL Tuning Advisor Recommendations:** Oracle's SQL Tuning Advisor analyzes SQL statements and provides recommendations for improving their performance, such as creating indexes, rewriting queries, or refreshing optimizer statistics.
- **Automatic Workload Repository (AWR) Reports:** AWR reports provide a comprehensive overview of database performance over a specified time period. They include detailed information on wait events, SQL statistics, system statistics, and other performance-related metrics.
- **Active Session History (ASH) Data:** ASH data captures detailed information about active sessions and their activities, including SQL statements executed, wait events encountered, and resource consumption. Analyzing ASH data helps identify performance bottlenecks in real-time.
- **Database Performance Baselines:** Establishing performance baselines allows you to compare current performance metrics against historical data to identify trends, anomalies, and deviations from normal behavior. This helps in proactively identifying and addressing performance issues.
- **Resource Usage Metrics:** Monitoring resource usage metrics such as CPU utilization, memory usage, disk I/O rates, and network traffic provides insights into system resource contention and helps identify potential performance bottlenecks.
- **Database Object Statistics:** Analyzing statistics related to database objects such as tables, indexes, and partitions helps identify poorly performing objects,

excessive fragmentation, and opportunities for optimization through index reorganization, partitioning, or table reorganization.

By leveraging these types of data and metrics, database administrators and performance tuning experts can gain valuable insights into the performance of Oracle databases, identify areas for improvement, and implement targeted optimizations to enhance overall system performance and efficiency.

Managing Memory

- Memory management plays a crucial role in Oracle performance management, as it directly impacts the efficiency and responsiveness of the database. Here's an overview of memory management in Oracle and its significance for performance:

System Global Area (SGA):

- The SGA is a shared memory region that stores data and control information for the Oracle instance.
- Components of the SGA include the database buffer cache, shared pool, large pool, redo log buffer, and Java pool.
- Proper sizing and configuration of the SGA are essential for optimizing database performance. Adequate memory allocation to each SGA component ensures efficient data caching, SQL statement execution, and transaction processing.

Program Global Area (PGA):

- The PGA is a memory region allocated for each Oracle session/process.
- It stores session-specific data such as SQL work areas, sort areas, and session variables.
- Optimizing PGA memory allocation involves configuring parameters such as PGA_AGGREGATE_TARGET to ensure that each session has sufficient memory for its operations without exhausting system resources.

Automatic Memory Management (AMM):

- Oracle provides features like Automatic Shared Memory Management (ASMM) and Automatic PGA Memory Management (APMM) to simplify memory management.
- ASMM dynamically adjusts the sizes of SGA components based on workload requirements, reducing the need for manual tuning.
- APMM dynamically allocates PGA memory for individual sessions based on their memory demands, optimizing memory usage across the database instance.

Memory Advisor:

- The Memory Advisor provides recommendations for optimizing memory allocation in Oracle databases.
- It analyzes database workload patterns and makes suggestions for adjusting SGA and PGA parameters to improve performance and resource utilization.

Monitoring and Tuning:

- Continuous monitoring of memory usage and performance metrics is essential for identifying memory-related bottlenecks and inefficiencies.
- Tools like Oracle Enterprise Manager (OEM), Automatic Workload Repository (AWR), and Statspack provide insights into memory utilization, buffer cache hit ratios, PGA memory usage, and other relevant metrics.
- Tuning memory-related parameters, such as SGA_TARGET, SGA_MAX_SIZE, and PGA_AGGREGATE_TARGET, based on workload characteristics and performance monitoring data, helps optimize memory utilization and overall database performance.

Shared Pool and Library Cache Management:

- Efficient management of the shared pool and library cache is critical for reducing parsing overhead and optimizing SQL statement execution.
- Techniques such as cursor sharing, SQL statement caching, and judicious use of bind variables help minimize memory contention and improve query performance.

Effective memory management in Oracle involves properly sizing and configuring the SGA and PGA, leveraging features like AMM, monitoring memory usage and performance metrics, and continuously tuning memory-related parameters to ensure optimal database performance and resource utilization.

Manage Private Temporary Tables

- Managing private temporary tables (PTTs) can contribute to performance tuning in Oracle databases by improving query performance, reducing contention, and optimizing resource usage.
- PTTs reside in memory (session's PGA), enabling quicker data access compared to traditional temporary tables stored on disk. This can significantly improve query performance, especially for operations involving frequent access to temporary data.

Use Cases for Private Temporary Tables:

- PTTs are temporary tables that are session-specific and are automatically dropped at the end of the session.
- They are useful for storing intermediate results, temporary data, or complex calculations within a session without affecting other sessions.
- Common use cases include complex data transformations, intermediate result sets, and temporary storage during complex query processing.

Optimal Creation and Usage of PTT:

- Create PTTs only when necessary and for the duration required within a session.
- Use PTTs for storing intermediate results that are reused multiple times within a session, avoiding repeated expensive computations or queries.
- Be mindful of the overhead associated with creating and dropping PTTs frequently, as it can impact performance. Consider reusing existing PTTs within a session whenever possible.
- Here are some considerations for managing PTTs effectively:

Indexing Considerations:

- Indexes can improve query performance on PTTs, especially for large datasets or complex queries.
- Evaluate the benefit of creating indexes on PTTs based on query patterns, data volume, and performance requirements.
- Avoid over-indexing PTTs, as it can increase overhead during data manipulation operations and degrade performance.

Monitoring and Management:

- Monitor the usage and resource consumption of PTTs using Oracle's performance monitoring tools, such as AWR reports, ASH data, and SQL monitoring.
- Identify PTT-related performance bottlenecks, such as excessive temporary space usage, contention, or inefficient query execution, and address them proactively.
- Regularly review and clean up unused or unnecessary PTTs to reclaim temporary tablespace and avoid resource wastage.

Temporary Tablespace Configuration:

- Ensure that the temporary tablespace is adequately sized and configured to accommodate the temporary space requirements of PTTs.
- Monitor temporary tablespace usage and growth over time, adjusting the size and configuration as needed to prevent performance degradation due to space constraints.
- Optimizing Query Performance:
- Design queries that leverage PTTs efficiently to reduce I/O and improve query performance.
- Consider using parallel processing, partitioning, or other optimization techniques to enhance query performance when working with large datasets or complex computations involving PTTs.

Session and Transaction Management:

- Properly manage sessions and transactions to ensure that PTTs are dropped at the end of the session or transaction to avoid resource leakage and contention.
- Be cautious when using PTTs within long-running transactions to minimize temporary space usage and prevent potential transactional conflicts.

By effectively managing private temporary tables in Oracle databases, you can optimize query performance, reduce resource contention, and improve overall database efficiency as part of your performance tuning efforts.

Create private temporary table statement

To create a new private temporary table, you use the `CREATE PRIVATE TEMPORARY TABLE` statement:

```
CREATE PRIVATE TEMPORARY TABLE table_name(
    column_definition,
    ...
) ON COMMIT [DROP DEFINITION | PRESERVE DEFINITION];
```

In this syntax:

- First, specify the name of the temporary table, which follows the naming rule mentioned above.
- Second, specify a list of columns with their definitions.
- Third, use the `ON COMMIT` clause to indicate whether the table is transaction-specific or session-specific:

The `ON COMMIT DROP DEFINITION` option creates a private temporary table that is transaction-specific. At the end of the transaction, Oracle drops both table definition and data.

The `ON COMMIT PRESERVE DEFINITION` option creates a private temporary table that is session-specific. Oracle removes all data and drops the table at the end of the session.

- By default, Oracle uses ON COMMIT DROP DEFINITION if you omit the ON COMMIT option

Creating a private temporary table example that is transaction-specific

- First, create a new temporary table that is transaction-specific:

```
CREATE PRIVATE TEMPORARY TABLE ora$ppt_temp1(
    id INT,
    description VARCHAR2(100)
) ON COMMIT DROP DEFINITION;
```

- Next, insert a row into the ora\$ppt_temp1 table:

```
INSERT INTO ora$ppt_temp1(id,description)
VALUES(1,'Transaction-specific private temp table');
```

- Then, view the contents of the ora\$ppt_temp1 table:

```
SELECT id, description
FROM ora$ppt_temp1;
```

- After that, commit the transaction:

```
COMMIT;
```

- Finally, query data from the ora\$ppt_temp1 table:

```
SELECT id, description
FROM ora$ppt_temp1;
```

- Oracle issued the following error because it already dropped the ora\$ppt_temp1 table at the end of the transaction.

```
ORA-00942: table or view does not exist
```